



RAJALAKSHMI ENGINEERING COLLEGE

Laboratory Record Note Book

NAME

BRANCH

UNIVERSITY REGISTER No.

COLLEGE ROLL No.

SEMESTER

ACADEMIC YEAR



BONAFIDE CERTIFICATE

NAME

ACADEMIC YEAR SEMESTER BRANCH.....

UNIVERSITY REGISTER No.

Certified that this is the bonafide record of work done by the above student in the

..... Laboratory during the year 20 - 20

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on

Internal Examiner

External Examiner

CB23731 – DATA VISUALIZATION TECHNIQUES**LAB RECORD – INDEX****Name:****Branch:****Roll No:**

DATE	EX. NO.	EXPERIMENT NAME	QR CODE	MARK	SIGNATURE
	1a	Setting Up Python Environment And Libraries – Jupyter Notebook			
	1b	EDA – Data Science Frameworks(CRISP-DM, SEMMA, KDD)			
	2	Exploratory Data Analysis (EDA) – Data Import and Export			
	3	Exploratory Data Analysis (EDA) – Data Cleaning and Data Transformation			
	4a	Exploratory Data Analysis (EDA) – Data Inspection, Filtering, Statistical Analysis and Correlation – Student data			
	4b	Exploratory Data Analysis (EDA) – Data Inspection, Filtering, Statistical Analysis and Correlation			
	5a	Data Visualization Using Matplotlib – Sales data			
	5b	Data Visualization Using Matplotlib			
	6a	Data Visualization Using Power BI - sales data			

	6b	Data Visualization Using Power BI – Vehicle Analysis Data			
	6c	Data Visualization Using Power BI – Vehicle Analysis			
	7a	Data Visualization Using Tableau – Student Performance			
	7b	Data Visualization Using Tableau – Vehicle Analysis			
	8	Mini Project – Healthcare Patient Analytics			
	9	Mini Project – Financial& Banking Analytics			

EX No:1a	SETTING UP PYTHON ENVIRONMENT AND LIBRARIES – JUPYTER NOTEBOOK

Aim:

To install Python, configure Jupyter Notebook, understand the notebook environment, execute Python programs, create Markdown cells, and use basic Jupyter Notebook features for Data Analytics applications.

Algorithm:**Algorithm 1: Installing Python**

1. Download Python from python.org
2. Run the installer
3. Select "Add Python to PATH"
4. Click Install Now
5. Verify installation using: python --version
6. Observe installed Python version

Algorithm 2: Installing Jupyter Notebook

1. Open Command Prompt
2. Install Jupyter pip install notebook
3. Launch Jupyter Notebook jupyter notebook
4. Browser automatically opens Notebook Dashboard.

Algorithm 3: Creating a Notebook

1. Open Jupyter Dashboard
2. Select New → Python 3
3. Create Notebook
4. Rename Notebook
5. Create Code and Markdown Cells
6. Execute cells using Shift + Enter

Program:**Program 1 — Simple Python Program**

```
print("Welcome to Data Analytics Lab")
```

Output:

Welcome to Data Analytics Lab

Program 2 — Arithmetic Operations

```
a = 20
```

```
b = 10
```

```
print("Addition =", a + b)
print("Subtraction =", a - b)
print("Multiplication =", a * b)
print("Division =", a / b)
```

Output:

```
Addition = 30
Subtraction = 10
Multiplication = 200
Division = 2.0
```

Program 3 — Display Student Details

```
name = "John"
dept = "AI&DS"
cgpa = 8.9
print("Name:", name)
print("Department:", dept)
print("CGPA:", cgpa)
```

Output:

```
Name: John
Department: AI&DS
CGPA: 8.9
```

Program 4 — Creating Markdown Cell

```
# Data Analytics Lab
### Jupyter Notebook

Then press Shift + Enter.
```

Program 5 — Using NumPy Library

```
import numpy as np
a = np.array([10, 20, 30, 40, 50])
print(a)
print("Mean =", np.mean(a))
print("Sum =", np.sum(a))
```

Output:

```
[10 20 30 40 50]
Mean = 30.0
Sum = 150
```

Program 6 — Using Pandas Library

```
import pandas as pd

data = {
    'Name': ['John', 'David', 'Alex'],
    'Marks': [85, 90, 95]
}

df = pd.DataFrame(data)

print(df)
```

Output:

	Name	Marks
0	John	85
1	David	90
2	Alex	95

Program 7 — Data Visualization

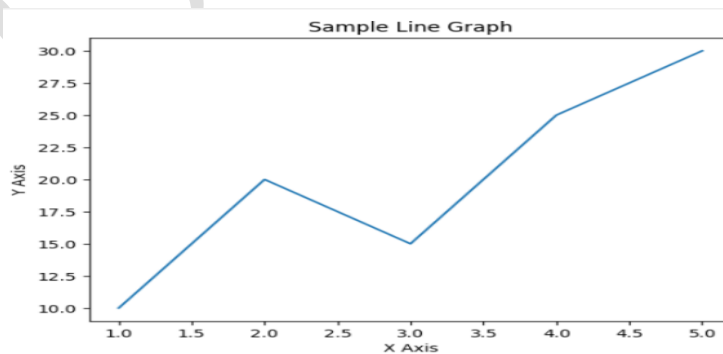
```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 25, 30]

plt.plot(x, y)

plt.title("Sample Line Graph")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.show()
```

Output:



Result:

EX No:1b	EDA DATA CLEANING

AIM

To perform exploratory data analysis on the Vehicle Analysis Dataset by examining the dataset structure, dimensions, data types, statistical characteristics, missing values, country-wise and region-wise attack distributions, attack types, correlations among numerical attributes, and exporting the dataset into different file formats.

ALGORITHM

1. Import the required libraries such as kagglehub, os, and pandas.
2. Download the Vehicle Analysis Dataset using KaggleHub.
3. Identify the CSV file available in the downloaded dataset.
4. Load the CSV file into a Pandas DataFrame using latin1 encoding.
5. Display the first five and last five records.
6. Display the dataset information, shape, column names, and data types.
7. Generate descriptive statistical information for numerical attributes.
8. Identify the number of missing values in each column.
9. Find the top 10 countries based on the number of terrorist attacks.
10. Analyze the distribution of attacks across different regions.
11. Analyze the different types of terrorist attacks.
12. Display the unique attack types available in the dataset.
13. Generate the correlation matrix for numerical attributes.
14. Export the dataset into CSV, Excel, and JSON formats.
15. Display a confirmation message after successful file creation.

CODE

```
import kagglehub

# Download latest version

path = kagglehub.dataset_download("nehalbirla/vehicle-dataset-from-cardekho")

print("Path to dataset files:", path)

import os

import pandas as pd

print("Dataset path:", path)

print("\nFiles in dataset:")

print(os.listdir(path))

# Find CSV file automatically
```

```
csv_file = [f for f in os.listdir(path) if f.endswith(".csv")][0]
print("\nCSV file:", csv_file)

# Load dataset
df = pd.read_csv(
    os.path.join(path, csv_file),
    low_memory=False
)

print("\nFirst 5 rows:")
display(df.head())
print("\nLast 5 rows:")
display(df.tail())

print("\nDataset information:")
df.info()
print("\nShape:")
print(df.shape)
print("\nColumns:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)
print("\nStatistical description:")
display(df.describe())
print("\nMissing values:")
display(df.isnull().sum())

print("\nTop 10 car names:")
display(df["Car_Name"].value_counts().head(10))
print("\nFuel types:")
display(df["Fuel_Type"].value_counts())
print("\nSeller types:")
display(df["Seller_Type"].value_counts())
```

```
print("\nTransmission types:")
display(df["Transmission"].value_counts())
print("\nUnique fuel types:")
print(df["Fuel_Type"].unique())
print("\nUnique seller types:")
print(df["Seller_Type"].unique())
```

```
print("\nUnique transmission types:")
print(df["Transmission"].unique())
print("\nUnique owner types:")
print(df["Owner"].unique())
print("\nCorrelation:")
display(df.corr(numeric_only=True))
```

```
# Excel
df.to_excel("vehicle_analysis.xlsx", index=False)
# JSON
df.to_json("vehicle_analysis.json", orient="records", indent=4)
# CSV
df.to_csv("vehicle_analysis.csv", index=False)
print("\nCSV, Excel and JSON files created successfully.")
```

OUTPUT

First 5 rows:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner
0	ritz	2014	3.35	5.59	27000	Petrol	Dealer	Manual	0
1	sx4	2013	4.75	9.54	43000	Diesel	Dealer	Manual	0
2	ciaz	2017	7.25	9.85	6900	Petrol	Dealer	Manual	0
3	wagon r	2011	2.85	4.15	5200	Petrol	Dealer	Manual	0
4	swift	2014	4.60	6.87	42450	Diesel	Dealer	Manual	0



Last 5 rows:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner
296	city	2016	9.50	11.6	33988	Diesel	Dealer	Manual	0
297	brio	2015	4.00	5.9	60000	Petrol	Dealer	Manual	0
298	city	2009	3.35	11.0	87934	Petrol	Dealer	Manual	0
299	city	2017	11.50	12.5	9000	Diesel	Dealer	Manual	0
300	brio	2016	5.30	5.9	5464	Petrol	Dealer	Manual	0

Dataset information:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 301 entries, 0 to 300

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	Car_Name	301 non-null	object
1	Year	301 non-null	int64
2	Selling_Price	301 non-null	float64
3	Present_Price	301 non-null	float64
4	Kms_Driven	301 non-null	int64
5	Fuel_Type	301 non-null	object
6	Seller_Type	301 non-null	object
7	Transmission	301 non-null	object
8	Owner	301 non-null	int64

dtypes: float64(2), int64(3), object(4)

memory usage: 21.3+ KB

Shape:
(301, 9)

Columns:

['Car_Name', 'Year', 'Selling_Price', 'Present_Price', 'Kms_Driven', 'Fuel_Type', 'Seller_Type', 'Transmission', 'Owner']

Data types:

```
Car_Name      object
Year          int64
Selling_Price float64
Present_Price float64
Kms_Driven    int64
Fuel_Type     object
Seller_Type   object
Transmission  object
Owner         int64
dtype: object
```

Statistical description:

	Year	Selling_Price	Present_Price	Kms_Driven	Owner
count	301.000000	301.000000	301.000000	301.000000	301.000000
mean	2013.627907	4.661296	7.628472	36947.205980	0.043189
std	2.891554	5.082812	8.644115	38886.883882	0.247915
min	2003.000000	0.100000	0.320000	500.000000	0.000000
25%	2012.000000	0.900000	1.200000	15000.000000	0.000000
50%	2014.000000	3.600000	6.400000	32000.000000	0.000000
75%	2016.000000	6.000000	9.900000	48767.000000	0.000000
max	2018.000000	35.000000	92.600000	500000.000000	3.000000

Missing values:

	0
Car_Name	0
Year	0
Selling_Price	0
Present_Price	0
Kms_Driven	0
Fuel_Type	0
Seller_Type	0
Transmission	0
Owner	0

dtype: int64

Top 10 car names:

	count
Car_Name	
city	26
corolla altis	16
verna	14
fortuner	11
brio	10

	count
Car_Name	
ciaz	9
i20	9
innova	9
grand i10	8
Royal Enfield Classic 350	7

dtype: int64

Unique fuel types:
['Petrol' 'Diesel' 'CNG']

Unique seller types:
['Dealer' 'Individual']

Unique transmission types:
['Manual' 'Automatic']

Unique owner types:
[0 1 3]

Correlation:

	Year	Selling_Price	Present_Price	Kms_Driven	Owner
Year	1.000000	0.236141	-0.047584	-0.524342	-0.182104
Selling_Price	0.236141	1.000000	0.878983	0.029187	-0.088344
Present_Price	-0.047584	0.878983	1.000000	0.203647	0.008057
Kms_Driven	-0.524342	0.029187	0.203647	1.000000	0.089216
Owner	-0.182104	-0.088344	0.008057	0.089216	1.000000

CSV, Excel and JSON files created successfully.

RESULT

Ex.no:2	EDA-DATA IMPORT AND EXPORT

Aim:

To inspect dataset features, detect null values, and export data into CSV, Excel, and SQLite database formats.

Algorithm:

1. Import pandas and sqlite3 libraries.
2. Read vehicle_analysis.csv using pd.read_csv().
3. Inspect data structure using .head(), .tail(), .info(), .describe(), and .isnull().sum().
4. Export DataFrame to .csv and .xlsx formats using .to_csv() and .to_excel().
5. Connect to hotel_booking.db using sqlite3 and write data into an SQL table via .to_sql().
6. Close the database connection and end execution.

Program:

```
import os
import pandas as pd
print("Dataset path:", path)
print("\nFiles in dataset:")
print(os.listdir(path))
# Find CSV file automatically
csv_file = [f for f in os.listdir(path) if f.endswith(".csv")][0]
print("\nCSV file:", csv_file)
# Load dataset
df = pd.read_csv(
    os.path.join(path, csv_file),
    low_memory=False
)
print("\nFirst 5 rows:")
display(df.head())
print("\nLast 5 rows:")
```

```
display(df.tail())
print("\nDataset information:")
df.info()
print("\nShape:")
print(df.shape)
print("\nColumns:")
print(df.columns.tolist())
print("\nData types:")
print(df.dtypes)
print("\nStatistical description:")
display(df.describe())
print("\nMissing values:")
display(df.isnull().sum())
print("\nTop 10 car names:")
display(df["Car_Name"].value_counts().head(10))
print("\nFuel types:")
display(df["Fuel_Type"].value_counts())
print("\nSeller types:")
display(df["Seller_Type"].value_counts())
print("\nTransmission types:")
display(df["Transmission"].value_counts())
print("\nUnique fuel types:")
print(df["Fuel_Type"].unique())
print("\nCorrelation:")
display(df.corr(numeric_only=True))
# Excel
df.to_excel("vehicle_analysis.xlsx", index=False)
# JSON
df.to_json("vehicle_analysis.json", orient="records", indent=4)
# CSV
df.to_csv("vehicle_analysis.csv", index=False)
print("\nCSV, Excel and JSON files created successfully.")
```

Output:

First 5 rows:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner
0	ritz	2014	3.35	5.59	27000	Petrol	Dealer	Manual	0
1	sx4	2013	4.75	9.54	43000	Diesel	Dealer	Manual	0
2	ciaz	2017	7.25	9.85	6900	Petrol	Dealer	Manual	0
3	wagon r	2011	2.85	4.15	5200	Petrol	Dealer	Manual	0
4	swift	2014	4.60	6.87	42450	Diesel	Dealer	Manual	0

Last 5 rows:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner
296	city	2016	9.50	11.6	33988	Diesel	Dealer	Manual	0
297	brio	2015	4.00	5.9	60000	Petrol	Dealer	Manual	0
298	city	2009	3.35	11.0	87934	Petrol	Dealer	Manual	0
299	city	2017	11.50	12.5	9000	Diesel	Dealer	Manual	0
300	brio	2016	5.30	5.9	5464	Petrol	Dealer	Manual	0

```
Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 301 entries, 0 to 300
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Car_Name        301 non-null   object
1   Year            301 non-null   int64
2   Selling_Price   301 non-null   float64
3   Present_Price   301 non-null   float64
4   Kms_Driven      301 non-null   int64
5   Fuel_Type       301 non-null   object
6   Seller_Type     301 non-null   object
7   Transmission    301 non-null   object
8   Owner          301 non-null   int64
dtypes: float64(2), int64(3), object(4)
memory usage: 21.3+ KB

Shape:
(301, 9)

Columns:
['Car_Name', 'Year', 'Selling_Price', 'Present_Price', 'Kms_Driven', 'Fuel_Type', 'Seller_Type', 'Transmission', 'Owner']
```

Statistical description:

	Year	Selling_Price	Present_Price	Kms_Driven	Owner
count	301.000000	301.000000	301.000000	301.000000	301.000000
mean	2013.627907	4.661296	7.628472	36947.205980	0.043189
std	2.891554	5.082812	8.644115	38886.883882	0.247915
min	2003.000000	0.100000	0.320000	500.000000	0.000000
25%	2012.000000	0.900000	1.200000	15000.000000	0.000000
50%	2014.000000	3.600000	6.400000	32000.000000	0.000000
75%	2016.000000	6.000000	9.900000	48767.000000	0.000000
max	2018.000000	35.000000	92.600000	500000.000000	3.000000

Result:

Ex.no:3	DATA CLEANING

Aim:

Algorithm:

Check Missing Values:

1. Start.
2. Import the pandas library.
3. Load the dataset into a DataFrame (df).
4. Display dataset information using df.info().
5. Check missing values using df.isnull().
6. Count missing values using df.isnull().sum().
7. Check missing values using df.isna().
8. Check non-missing values using df.notnull().
9. Display the results.

Drop Missing Values

1. Start.
2. Import the pandas library.
3. Load the dataset into a DataFrame (df).
4. Drop missing rows using dropna().
5. Drop rows (axis=0) or columns (axis=1).
6. Use how='any' to remove rows/columns containing any missing value.
7. Use how='all' to remove rows/columns only if all values are missing.
8. Use thresh=N to keep rows/columns with at least N non-missing values.
9. Use subset=['column_name'] to drop rows based on specific columns.
10. Use inplace=True to modify the original DataFrame.
11. Display the cleaned DataFrame.

Program:

```
import pandas as pd
import os
import kagglehub

# Download latest version
path = kagglehub.dataset_download("nehalbirla/vehicle-dataset-from-cardekho")
print("Path to dataset files:", path)

# Load the vehicle dataset already downloaded using kagglehub
csv_file = [f for f in os.listdir(path) if f.endswith(".csv")][0]
df = pd.read_csv(
    os.path.join(path, csv_file),
    encoding="latin1",
    low_memory=False
)

# Clean column names
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")

# Display selected vehicle dataset columns
data = {
    "Year": df["year"].head(5).tolist(),
    "Car_Name": df["car_name"].head(5).tolist(),
    "Selling_Price": df["selling_price"].head(5).tolist(),
    "Fuel_Type": df["fuel_type"].head(5).tolist()
}

vehicle_df = pd.DataFrame(data)
print("Original DataFrame:")
print(vehicle_df)
print("\n1. First 5 rows (head):")
print(vehicle_df.head())
print("\n2. Last 2 rows (tail):")
print(vehicle_df.tail(2))
```

```

print("\n3. Shape (rows, columns):")
print(vehicle_df.shape)
print("\n4. Column Names:")
print(vehicle_df.columns)
print("\n5. DataFrame Information:")
vehicle_df.info()
print("\n6. Statistical Summary:")
print(vehicle_df.describe())
print("\n7. Vehicles after 2010:")
print(
    df[df["year"] > 2010][
        ["year", "car_name", "selling_price", "fuel_type"]
    ].head()
)

```

Output:

Path to dataset files: /kaggle/input/vehicle-dataset-from-cardekho

Original DataFrame:

	Year	Car_Name	Selling_Price	Fuel_Type
0	2014	ritz	3.35	Petrol
1	2013	sx4	4.75	Diesel
2	2017	ciaz	7.25	Petrol
3	2011	wagon r	2.85	Petrol
4	2014	swift	4.60	Diesel

1. First 5 rows (head):

	Year	Car_Name	Selling_Price	Fuel_Type
0	2014	ritz	3.35	Petrol
1	2013	sx4	4.75	Diesel
2	2017	ciaz	7.25	Petrol
3	2011	wagon r	2.85	Petrol
4	2014	swift	4.60	Diesel

2. Last 2 rows (tail):

	Year	Car_Name	Selling_Price	Fuel_Type
3	2011	wagon r	2.85	Petrol
4	2014	swift	4.60	Diesel

3. Shape (rows, columns):

(5, 4)

4. Column Names:

Index(['Year', 'Car_Name', 'Selling_Price', 'Fuel_Type'], dtype='object')

5. DataFrame Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 5 entries, 0 to 4

Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	Year	5 non-null	int64
1	Car_Name	5 non-null	object
2	Selling_Price	5 non-null	float64
3	Fuel_Type	5 non-null	object

dtypes: float64(1), int64(1), object(2)

memory usage: 292.0+ bytes

6. Statistical Summary:

	Year	Selling_Price
count	5.000000	5.000000
mean	2013.800000	4.560000
std	2.167948	1.707484
min	2011.000000	2.850000
25%	2013.000000	3.350000
50%	2014.000000	4.600000

75% 2014.000000 4.750000

max 2017.000000 7.250000

7. Vehicles after 2010:

	year	car_name	selling_price	fuel_type
0	2014	ritz	3.35	Petrol
1	2013	sx4	4.75	Diesel
2	2017	ciaz	7.25	Petrol
3	2011	wagon r	2.85	Petrol
4	2014	swift	4.60	Diesel

Result:

Ex.no:4a	EXPLORATORY DATA ANALYSIS (EDA) – DATA INSPECTION, FILTERING, STATISTICAL ANALYSIS AND CORRELATION

Aim:

To perform data inspection, filtering, grouping, aggregation, descriptive statistical analysis, and correlation analysis using Pandas and visualize the results using Matplotlib.

Algorithm:

1. Import the required Pandas, NumPy and Matplotlib libraries.
2. Read the stud_data.csv file using Pandas.
3. Display the dataset and inspect the first few records.
4. Find the number of rows and columns and display the data types.
5. Generate the statistical summary of the dataset.
6. Filter the students based on marks and attendance conditions.
7. Sort the student records based on marks and attendance.
8. Group the data based on department and calculate average marks and attendance.
9. Calculate mean, median, minimum, maximum and standard deviation of marks.
10. Calculate the correlation between marks and attendance.
11. Create bar chart, line chart, scatter plot, histogram and box plot using Matplotlib.
12. Display the results and interpret the obtained data.

Program:

```
# Import libraries

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

df = pd.read_csv("stud_data.csv")

# Display dataset

print(df)

# Display first five records

print(df.head())

# Display shape

print(df.shape)
```

```
print(df.dtypes)

print(df.describe())

print(df[df["Marks"] > 80])
# Filter students with attendance greater than 90

print(df[df["Attendance"] > 90])

# Sort students by marks

print(df.sort_values("Marks", ascending=False))

# Sort students by attendance

print(df.sort_values("Attendance", ascending=False))

# Department-wise average marks

print(df.groupby("Department")["Marks"].mean())

# Department-wise average attendance

print(df.groupby("Department")["Attendance"].mean())

# Calculate mean

print("Mean Marks:", df["Marks"].mean())

# Calculate median

print("Median Marks:", df["Marks"].median())

# Calculate minimum

print("Minimum Marks:", df["Marks"].min())

# Calculate maximum

print("Maximum Marks:", df["Marks"].max())

# Calculate standard deviation

print("Standard Deviation:", df["Marks"].std())

# Display correlation matrix

print(df[["Marks", "Attendance"]].corr())
```

```
# Display correlation between marks and attendance
print("Correlation:", df["Marks"].corr(df["Attendance"]))

# Bar chart
plt.bar(df["Name"], df["Marks"])

plt.title("Student Marks")

plt.xlabel("Name")

plt.ylabel("Marks")

plt.show()

# Line chart
plt.plot(df["Name"], df["Attendance"], marker="o")

plt.title("Student Attendance")

plt.xlabel("Name")

plt.ylabel("Attendance")

plt.show()

# Scatter plot
plt.scatter(df["Attendance"], df["Marks"])

plt.title("Marks vs Attendance")

plt.xlabel("Attendance")

plt.ylabel("Marks")

plt.show()

# Histogram
plt.hist(df["Marks"], bins=5)

plt.title("Marks Distribution")

plt.xlabel("Marks")

plt.ylabel("Number of Students")

plt.show()
```

```
# Box plot

plt.boxplot(df["Marks"])

plt.title("Marks Box Plot")

plt.ylabel("Marks")

plt.show()
```

Output:

	Roll No	Name	Department	Marks	Attendance
0	101	Arun	CSE	85	92
1	102	Bala	IT	78	88
2	103	Divya	CSE	95	96
3	104	Priya	AI&DS	90	94
4	105	Ravi	IT	72	85
5	106	Karthik	AI&DS	88	91

	Roll No	Name	Department	Marks	Attendance
0	101	Arun	CSE	85	92
1	102	Bala	IT	78	88
2	103	Divya	CSE	95	96
3	104	Priya	AI&DS	90	94
4	105	Ravi	IT	72	85

(6, 5)

```
Roll No      int64
Name         object
Department   object
Marks        int64
Attendance   int64
dtype: object
```

	Roll No	Marks	Attendance
count	6.000000	6.000000	6.00
mean	103.500000	84.666667	91.00
std	1.870829	8.382521	4.00
min	101.000000	72.000000	85.00
25%	102.250000	79.750000	88.75
50%	103.500000	86.500000	91.50
75%	104.750000	89.500000	93.50
max	106.000000	95.000000	96.00

	Roll No	Name	Department	Marks	Attendance
0	101	Arun	CSE	85	92
2	103	Divya	CSE	95	96
3	104	Priya	AI&DS	90	94
5	106	Karthik	AI&DS	88	91

	Roll No	Name	Department	Marks	Attendance
0	101	Arun	CSE	85	92
2	103	Divya	CSE	95	96
3	104	Priya	AI&DS	90	94
5	106	Karthik	AI&DS	88	91

	Roll No	Name	Department	Marks	Attendance
2	103	Divya	CSE	95	96
3	104	Priya	AI&DS	90	94
5	106	Karthik	AI&DS	88	91
0	101	Arun	CSE	85	92
1	102	Bala	IT	78	88
4	105	Ravi	IT	72	85

	Roll No	Name	Department	Marks	Attendance
2	103	Divya	CSE	95	96
3	104	Priya	AI&DS	90	94
0	101	Arun	CSE	85	92
5	106	Karthik	AI&DS	88	91
1	102	Bala	IT	78	88
4	105	Ravi	IT	72	85

Department

AI&DS 89.0

CSE 90.0

IT 75.0

Name: Marks, dtype: float64

Department

AI&DS 92.5

CSE 94.0

IT 86.5

Name: Attendance, dtype: float64

Mean Marks: 84.66666666666667

Median Marks: 86.5

Minimum Marks: 72

Maximum Marks: 95

Standard Deviation: 8.382521498133285

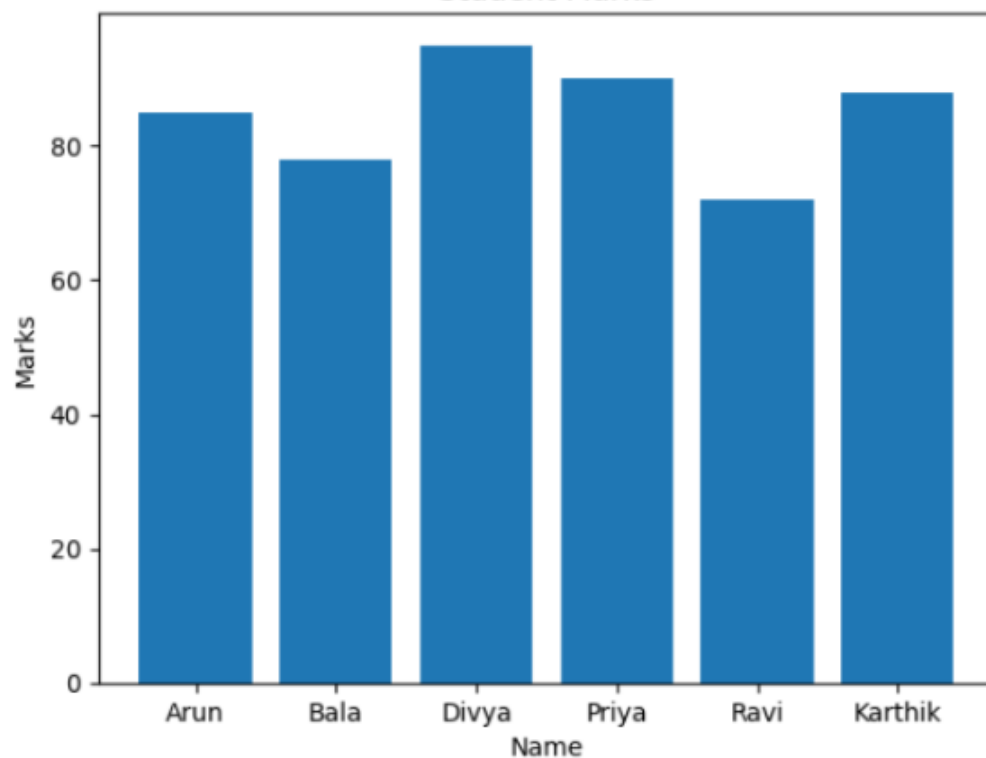
Marks Attendance

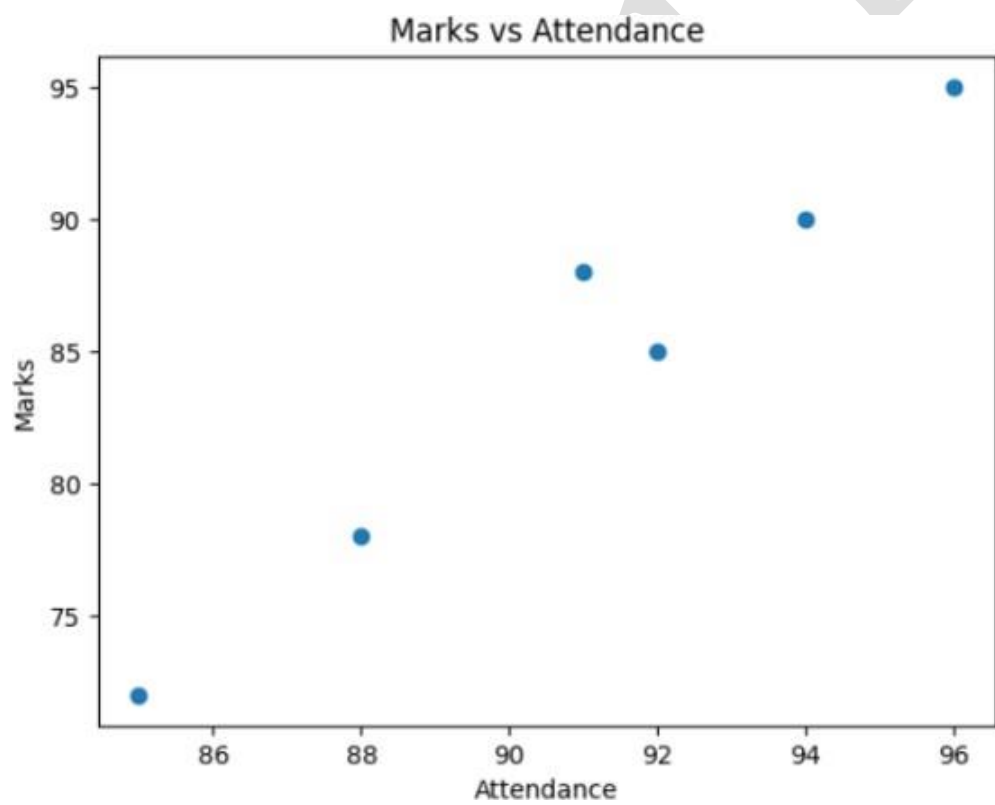
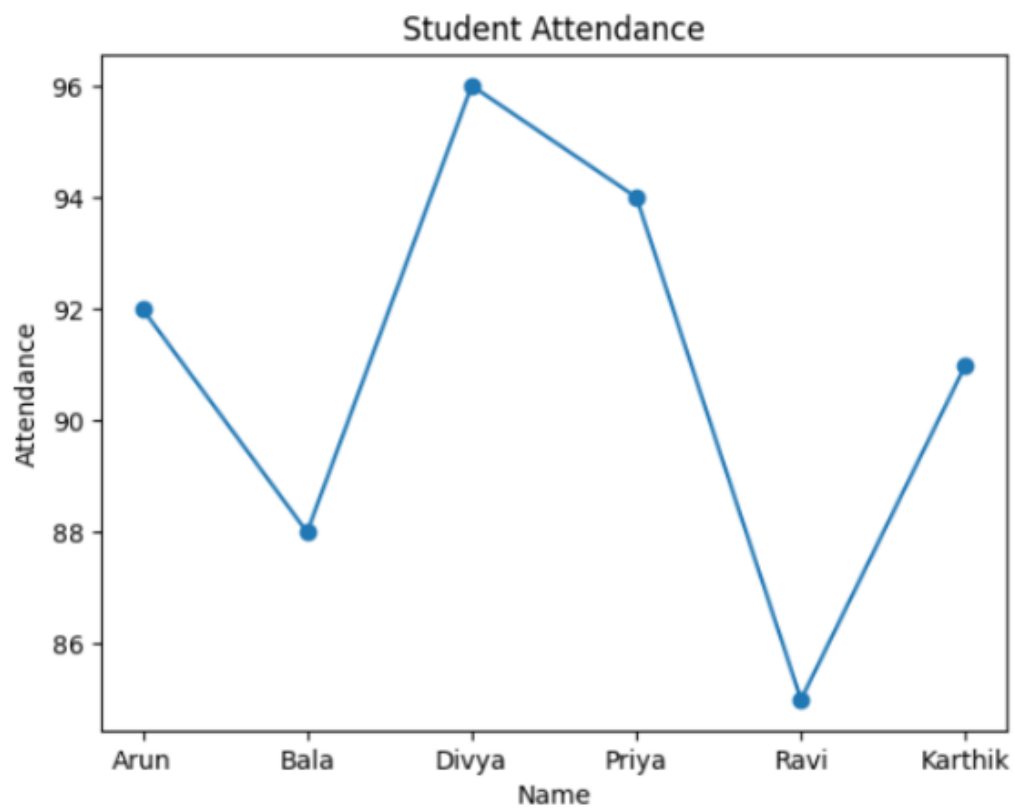
Marks 1.000000 0.978226

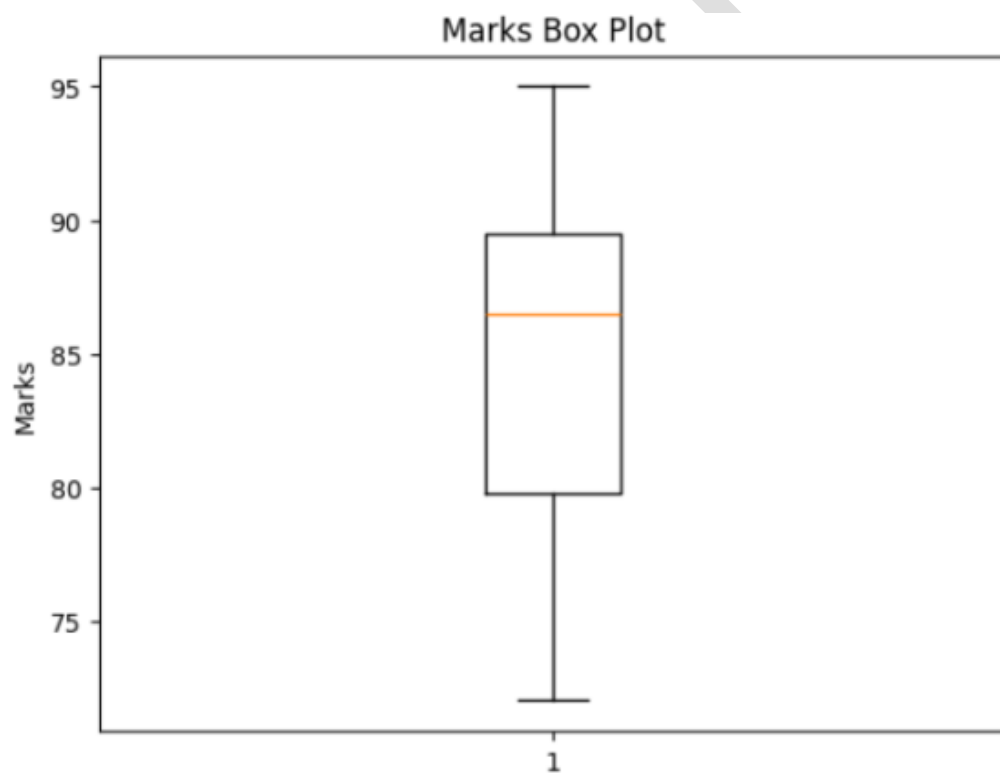
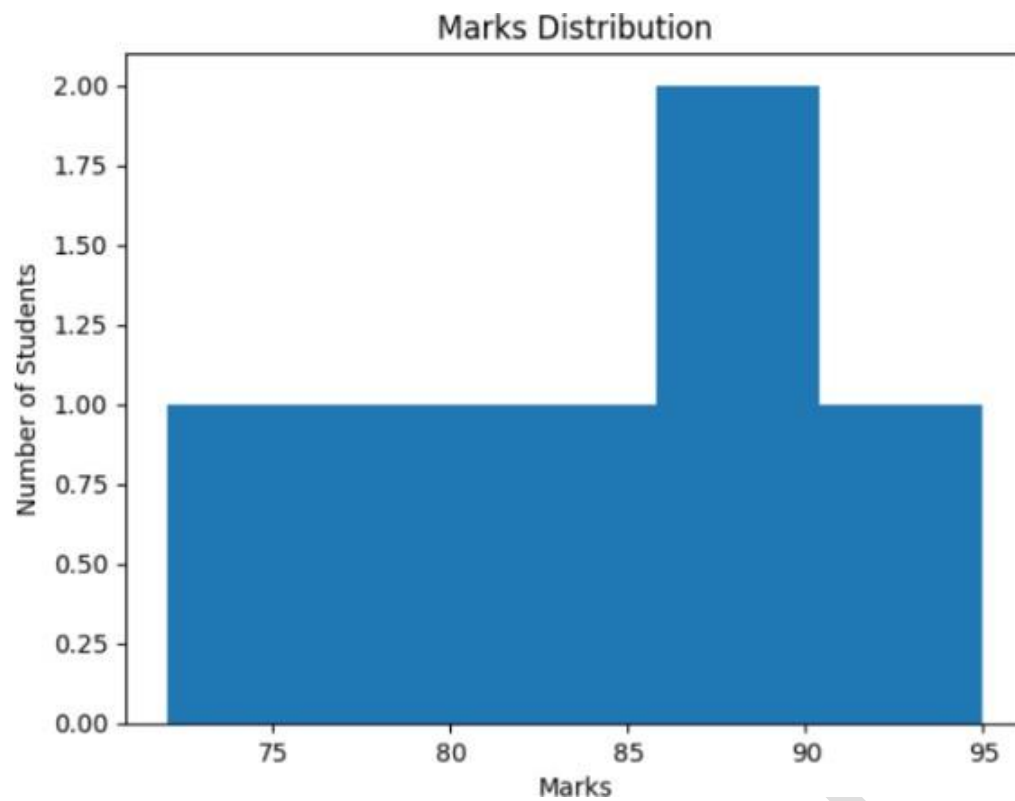
Attendance 0.978226 1.000000

Correlation: 0.9782259433305447

Student Marks







Result:

Ex.no:4b	EXPLORATORY DATA ANALYSIS (EDA) – DATA INSPECTION, FILTERING, STATISTICAL ANALYSIS AND CORRELATION

AIM

To perform data inspection, filtering, sorting, grouping, aggregation, descriptive statistical analysis, and correlation analysis on the Vehicle Analysis dataset using Pandas and visualize the analytical results using Matplotlib.

ALGORITHM

Algorithm 1: Dataset Inspection

1. Import the Vehicle Analysis dataset.
2. Display the first few records of the dataset.
3. Check the dimensions of the dataset.
4. Display the data types of the attributes.
5. Generate a statistical summary of the numerical attributes.

Algorithm 2: Filtering Data

1. Select the required attribute.
2. Specify a suitable filtering condition.
3. Apply the condition to the dataset.
4. Display the filtered records.

Algorithm 3: Sorting Data

1. Select the attribute to be sorted.
2. Apply the `sort_values()` function.
3. Specify the sorting order.
4. Display the sorted dataset.

Algorithm 4: Statistical Analysis

1. Select the required numerical attribute.
2. Calculate the mean.
3. Calculate the median.
4. Calculate the mode.
5. Calculate the standard deviation.
6. Display the calculated statistical measures.

Algorithm 5: Grouping and Aggregation

1. Select a categorical attribute such as region or country.
2. Apply the `groupby()` function.
3. Select the required numerical attribute.

4. Calculate an aggregate measure such as mean or sum.
5. Display the grouped results.

Algorithm 6: Correlation Analysis

1. Select the required numerical attributes.
2. Apply the corr() function.
3. Generate the correlation matrix.
4. Analyze the correlation coefficient between the selected attributes.
5. Visualize the relationship using an appropriate graph.

CODE

```
import kagglehub

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

import os

# =====
# LOAD DATASET FROM KAGGLE
# =====
path = kagglehub.dataset_download("nehalbirla/vehicle-dataset-from-cardekho")
print("Dataset Path:")
print(path)

# Find CSV file automatically

csv_files = [f for f in os.listdir(path) if f.endswith(".csv")]
print("\nCSV Files Found:")

print(csv_files)

csv_file = csv_files[0]

# Load Dataset

df = pd.read_csv(

    os.path.join(path, csv_file),

    encoding="latin1",
    low_memory=False
```

```

)

# Clean column names

df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")

print("\nDataset Loaded Successfully")

print("\nColumn Names:")

print(df.columns.tolist())

# =====
# PROGRAM 1 - DATA INSPECTION
# =====

print("\nFirst 5 Records")

print(df.head())

print("\nLast 5 Records")

print(df.tail())
# =====
# PROGRAM 2 - SHAPE
# =====

print("\nShape of Dataset")

print(df.shape)
print("\nRows =", df.shape[0])

print("\nColumns =", df.shape[1])
# =====
# PROGRAM 3 - DATA TYPES
# =====

print("\nData Types")
print(df.dtypes)
print("\nDataset Info")
df.info()

# =====
# PROGRAM 4 - FILTERING
# Selling Price > 10 Lakhs
# =====

```

```
high_price = df[df["selling_price"] > 1000000]
print("\nVehicles with Selling Price Greater Than 10 Lakhs")
```

```
print(
    high_price[
        [
            "car_name",
            "year",
            "selling_price",
            "present_price",
            "fuel_type"
        ]
    ].head(10)
)
```

```
# =====
# PROGRAM 5 - FILTER VEHICLES WITH AUTOMATIC TRANSMISSION
# =====
```

```
automatic = df[df["transmission"].str.lower() == "automatic"]
```

```
print("\nVehicles with Automatic Transmission")
```

```
print(
    automatic[
        [
            "car_name",
            "year",
            "selling_price",
            "fuel_type",

```

```

        "transmission"

    ]

].head(10)

)

# =====
# PROGRAM 6 - SORTING
# Sort by Selling Price
# =====

sorted_df = df.sort_values(

    by="selling_price",

    ascending=False

print("\nTop Vehicles by Selling Price")

print(

    sorted_df[

        [

            "car_name",

            "year",

            "selling_price",

            "present_price",

            "kms_driven"

        ]

    ].head(10)

)

# =====
# BAR CHART
# TOP 10 VEHICLES BY SELLING PRICE
# =====

```

```
top10 = sorted_df.head(10)
```

```
plt.figure(figsize=(10, 5))
```

```
plt.bar(
```

```
    range(len(top10)),
```

```
    top10["selling_price"]
```

```
)
```

```
plt.xticks(
```

```
    range(len(top10)),
```

```
    top10["car_name"],
```

```
    rotation=90
```

```
)
```

```
plt.xlabel("Car Name")
```

```
plt.ylabel("Selling Price")
```

```
plt.title("Top 10 Vehicles by Selling Price")
```

```
plt.show()
```

```
# =====
```

```
# PROGRAM 7 - STATISTICAL ANALYSIS
```

```
# =====
```

```
print("\nMean Selling Price =")
```

```
print(df["selling_price"].mean())
```

```
print("\nMedian Selling Price =")
```

```
print(df["selling_price"].median())
```

```
print("\nMode Selling Price =")
```

```
print(df["selling_price"].mode())
```

```
print("\nStandard Deviation =")
```

```
print(df["selling_price"].std())
```

```
print("\nMinimum Selling Price =")
```

```
print(df["selling_price"].min())
```

```
print("\nMaximum Selling Price =")
```

```
print(df["selling_price"].max())
```

```
# =====
```

```
# PROGRAM 8 - DESCRIPTIVE STATISTICS
```

```
# =====
```

```
print("\nDescriptive Statistics")
```

```
print(
```

```
    df[
```

```
        [
```

```
            "year",
```

```
            "selling_price",
```

```
            "present_price",
```

```
            "kms_driven"
```

```
        ]
```

```
    ].describe()
```

```
# =====
```

```
# PROGRAM 9 - GROUPING
```

```
# Average Selling Price by Fuel Type
```

```
# =====
```

```
group = df.groupby(
```

```
"fuel_type"

)["selling_price"].mean().sort_values(ascending=False)

print("\nAverage Selling Price by Fuel Type")

print(group)

# Visualization

group.plot(

    kind="bar",

    figsize=(10, 5)

)

plt.xlabel("Fuel Type")

plt.ylabel("Average Selling Price")

plt.title("Average Selling Price by Fuel Type")

plt.xticks(rotation=0)

plt.show()

# =====
# PROGRAM 10 - AGGREGATION
# =====

summary = df.groupby("fuel_type").agg({

    "selling_price": "mean",

    "present_price": "mean",

    "kms_driven": "mean"

})

print("\nFuel Type Aggregation")
```

```

print(summary)

# =====
# PROGRAM 11 - CORRELATION
# =====

numeric = df.select_dtypes(

    include=np.number

)

corr = numeric.corr()

print("\nCorrelation Matrix")

print(corr)

plt.figure(figsize=(10, 7))

sns.heatmap(

    corr,

    annot=True,

    cmap="coolwarm"

)

plt.title("Vehicle Dataset Correlation Heatmap")

plt.show()

# =====
# PROGRAM 12 - SCATTER PLOT
# Year vs Selling Price
# =====

```

```
plt.figure(figsize=(7, 5))
```

```
plt.scatter(  
    df["year"],  
    df["selling_price"]  
)
```

```
plt.xlabel("Year")
```

```
plt.ylabel("Selling Price")
```

```
plt.title("Year vs Selling Price")
```

```
plt.show()
```

```
# =====  
# PROGRAM 13 - HISTOGRAM  
# Selling Price Distribution  
# =====
```

```
plt.figure(figsize=(7, 5))
```

```
plt.hist(  
    df["selling_price"].dropna(),  
    bins=30  
)
```

```
plt.xlabel("Selling Price")
```

```
plt.ylabel("Frequency")
```

```
plt.title("Distribution of Vehicle Selling Prices")
```

```
plt.show()
```

```
# =====
# PROGRAM 14 - BOXPLOT
# =====

plt.figure(figsize=(7, 5))

sns.boxplot(

    x=df["selling_price"].dropna()

)

plt.title("Selling Price Boxplot")

plt.xlabel("Selling Price")
plt.show()
# =====
# PROGRAM 15 - VALUE COUNTS
# =====

print("\nFuel Type Counts")

print(

    df["fuel_type"].value_counts()

)

# =====
# PROGRAM 16 - UNIQUE VALUES
# =====

print("\nUnique Car Names")

print(

    df["car_name"].unique()

)

print("\nUnique Fuel Types")

print(

    df["fuel_type"].unique()
```

```
)  
print("\nUnique Seller Types")
```

```
print(  
    df["seller_type"].unique()  
)
```

```
print("\nUnique Transmissions")
```

```
print(  
    df["transmission"].unique()  
)
```

```
# =====  
# PROGRAM 17 - TOP 10 CAR NAMES  
# =====
```

```
print("\nTop 10 Car Names by Number of Vehicles")
```

```
car_counts = (  
    df["car_name"]  
    .value_counts()  
    .head(10)  
)
```

```
print(car_counts)
```

```
# =====  
# PROGRAM 18 - VEHICLES BY YEAR  
# =====
```

```
print("\nNumber of Vehicles by Year")
```

```
year_counts = (  
    df["year"]  
    .value_counts()  
    .sort_index()  
)
```

```

print(year_counts)

# =====
# PROGRAM 19 - TOP 10 CAR NAMES BY SELLING PRICE
# =====

print("\nTop 10 Car Names by Total Selling Price")

car_price = (

    df.groupby("car_name")["selling_price"]

    .sum()

    .sort_values(ascending=False)

    .head(10)

)
print(car_price)

# =====
# PROGRAM 20 - MOST COMMON FUEL TYPES
# =====

print("\nMost Common Fuel Types")

fuel_types = (

    df["fuel_type"]

    .value_counts()

    .head(10)

)
print(fuel_types)

# =====
# PROGRAM 21 - EXPORT DATA
# =====

df.to_csv(

    "vehicle_data.csv",

```

```

        index=False

    )
    df.to_excel(

        "vehicle_data.xlsx",

        index=False

    )
    df.to_json(

        "vehicle_data.json",

        orient="records",

        indent=4

    )
    print("\nCSV, Excel and JSON files exported successfully.")

# =====
# FINAL REPORT
# =====

print("\n=====")

print("EXPERIMENT COMPLETED SUCCESSFULLY")

print("=====")

print("Dataset Shape :", df.shape)

print(

    "Missing Values :",

    df.isnull().sum().sum()

)
print(

    "Total Vehicles :",

    len(df)

```

```
print(
    "Total Selling Price :",
    df["selling_price"].sum()
)
print(
    "Average Selling Price :",
    df["selling_price"].mean()
)
print(
    "Number of Car Names :",
    df["car_name"].nunique()
)
print(
    "Number of Fuel Types :",
    df["fuel_type"].nunique()
)
print(
    "Number of Seller Types :",
    df["seller_type"].nunique()
)
print(
    "Number of Transmissions :",
    df["transmission"].nunique()
)
```

```
print("Correlation Matrix Generated")
```

```
print("=====")
```

OUTPUT:

Dataset Path:
/kaggle/input/vehicle-dataset-from-cardekho

CSV Files Found:
['car data.csv', 'car details v4.csv', 'CAR DETAILS FROM CAR DEKHO.csv', 'Car details v3.csv']

Dataset Loaded Successfully

Column Names:
['car_name', 'year', 'selling_price', 'present_price', 'kms_driven', 'fuel_type', 'seller_type', 'transmission', 'owner']

First 5 Records

	car_name	year	selling_price	present_price	kms_driven	fuel_type
0	ritz	2014	3.35	5.59	27000	Petrol
1	sx4	2013	4.75	9.54	43000	Diesel
2	ciaz	2017	7.25	9.85	6900	Petrol
3	wagon r	2011	2.85	4.15	5200	Petrol
4	swift	2014	4.60	6.87	42450	Diesel

	seller_type	transmission	owner
0	Dealer	Manual	0
1	Dealer	Manual	0
2	Dealer	Manual	0
3	Dealer	Manual	0
4	Dealer	Manual	0

Last 5 Records

	car_name	year	selling_price	present_price	kms_driven	fuel_type
296	city	2016	9.50	11.6	33988	Diesel
297	brio	2015	4.00	5.9	60000	Petrol
298	city	2009	3.35	11.0	87934	Petrol
299	city	2017	11.50	12.5	9000	Diesel
300	brio	2016	5.30	5.9	5464	Petrol

	seller_type	transmission	owner
296	Dealer	Manual	0
297	Dealer	Manual	0
298	Dealer	Manual	0
299	Dealer	Manual	0
300	Dealer	Manual	0

Shape of Dataset
(301, 9)

Rows = 301
Columns = 9

Data Types

```
car_name      object
year          int64
selling_price  float64
present_price float64
kms_driven    int64
fuel_type     object
seller_type   object
transmission  object
owner         int64
dtype: object
```

Dataset Info

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 301 entries, 0 to 300

Data columns (total 9 columns):

```
#   Column      Non-Null Count  Dtype
--  --
```

```
0   car_name    301 non-null   object
1   year        301 non-null   int64
2   selling_price 301 non-null   float64
3   present_price 301 non-null   float64
4   kms_driven  301 non-null   int64
5   fuel_type    301 non-null   object
6   seller_type  301 non-null   object
7   transmission 301 non-null   object
8   owner        301 non-null   int64
```

dtypes: float64(2), int64(3), object(4)

memory usage: 21.3+ KB

Vehicles with Selling Price Greater Than 10 Lakhs

Empty DataFrame

Columns: [car_name, year, selling_price, present_price, fuel_type]

Index: []

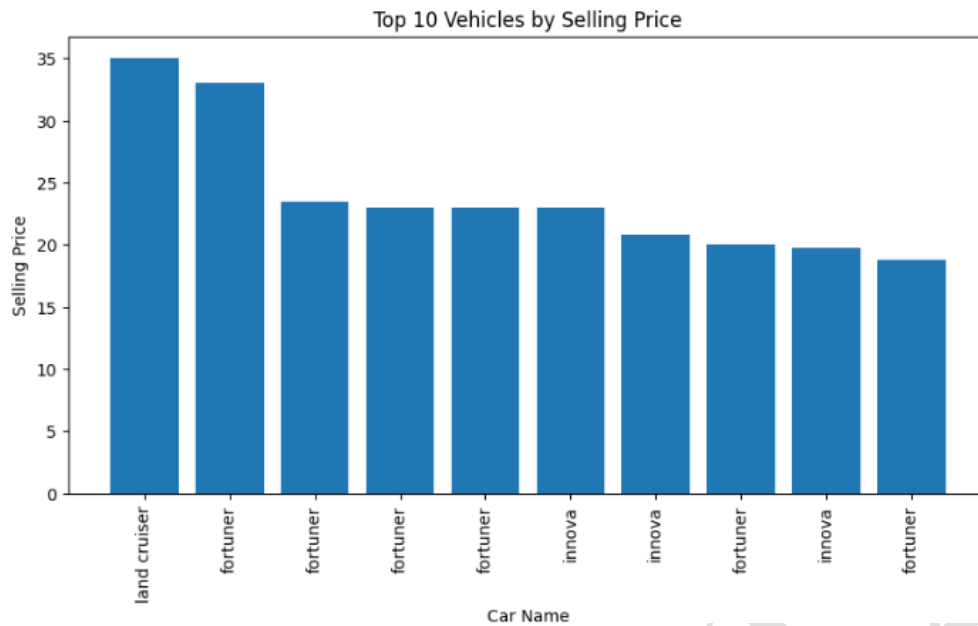
Vehicles with Automatic Transmission

	car_name	year	selling_price	fuel_type	transmission
12	ciaz	2015	7.50	Petrol	Automatic
22	sx4	2011	4.40	Petrol	Automatic
40	baleno	2016	5.85	Petrol	Automatic
45	ciaz	2014	7.50	Petrol	Automatic
49	ciaz	2017	7.75	Petrol	Automatic
50	fortuner	2012	14.90	Diesel	Automatic
51	fortuner	2015	23.00	Diesel	Automatic
52	innova	2017	18.00	Diesel	Automatic
53	fortuner	2013	16.00	Diesel	Automatic
55	corolla altis	2009	3.60	Petrol	Automatic

Top Vehicles by Selling Price

	car_name	year	selling_price	present_price	kms_driven
86	land cruiser	2010	35.00	92.60	78000
64	fortuner	2017	33.00	36.23	6000
63	fortuner	2015	23.50	35.96	47000
51	fortuner	2015	23.00	30.61	40000
93	fortuner	2015	23.00	30.61	40000
82	innova	2017	23.00	25.39	15000
96	innova	2016	20.75	25.39	29000

59	fortuner	2014	19.99	35.96	41000
66	innova	2017	19.75	23.15	11000
62	fortuner	2014	18.75	35.96	78000



Mean Selling Price =
4.661295681063123

Median Selling Price =
3.6

Mode Selling Price =
0 0.45
1 0.60

Name: selling_price, dtype: float64

Standard Deviation =
5.082811556177804

Minimum Selling Price =
0.1

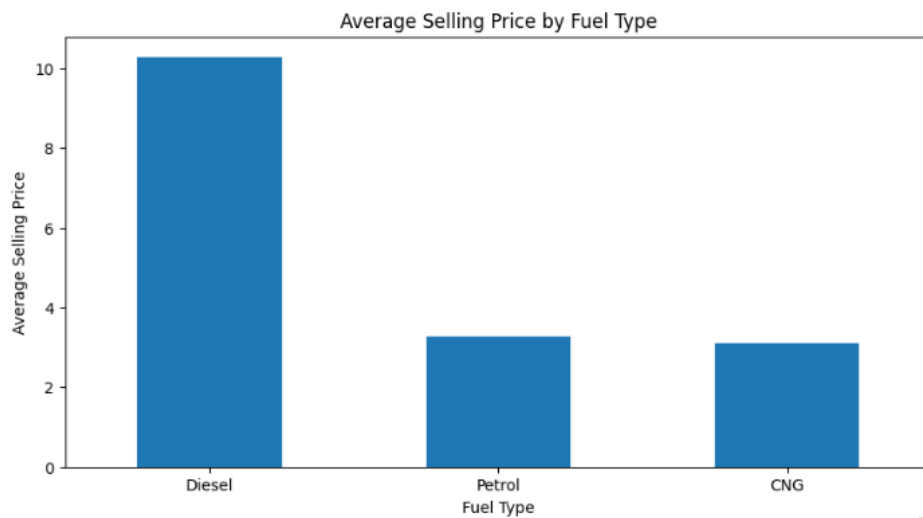
Maximum Selling Price =
35.0

Descriptive Statistics

	year	selling_price	present_price	kms_driven
count	301.000000	301.000000	301.000000	301.000000
mean	2013.627907	4.661296	7.628472	36947.205980
std	2.891554	5.082812	8.644115	38886.883882
min	2003.000000	0.100000	0.320000	500.000000
25%	2012.000000	0.900000	1.200000	15000.000000
50%	2014.000000	3.600000	6.400000	32000.000000
75%	2016.000000	6.000000	9.900000	48767.000000
max	2018.000000	35.000000	92.600000	500000.000000

Average Selling Price by Fuel Type
fuel_type

Diesel 10.278500
Petrol 3.264184
CNG 3.100000
Name: selling_price, dtype: float64

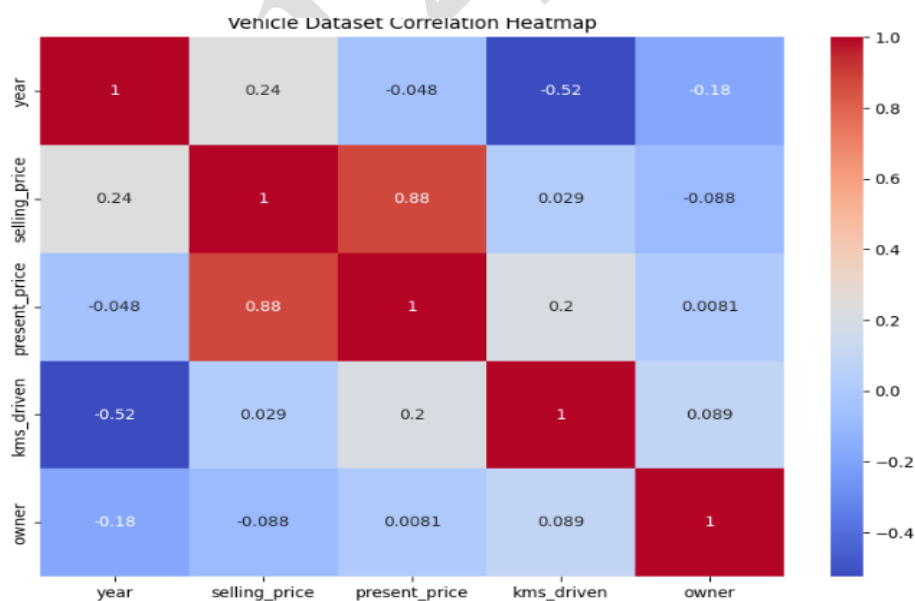


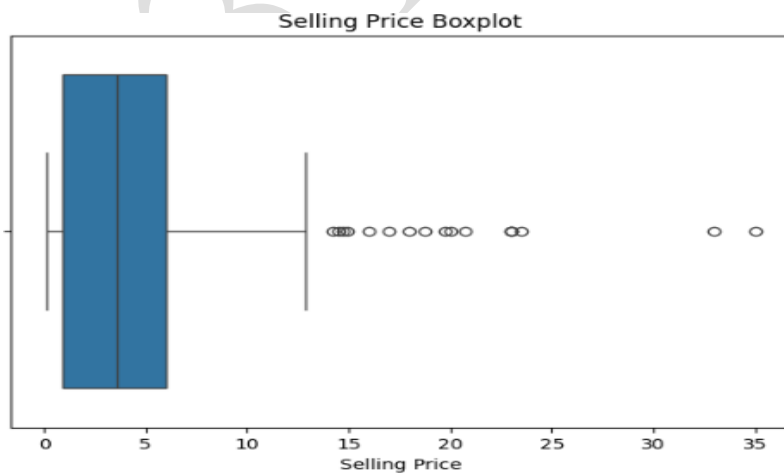
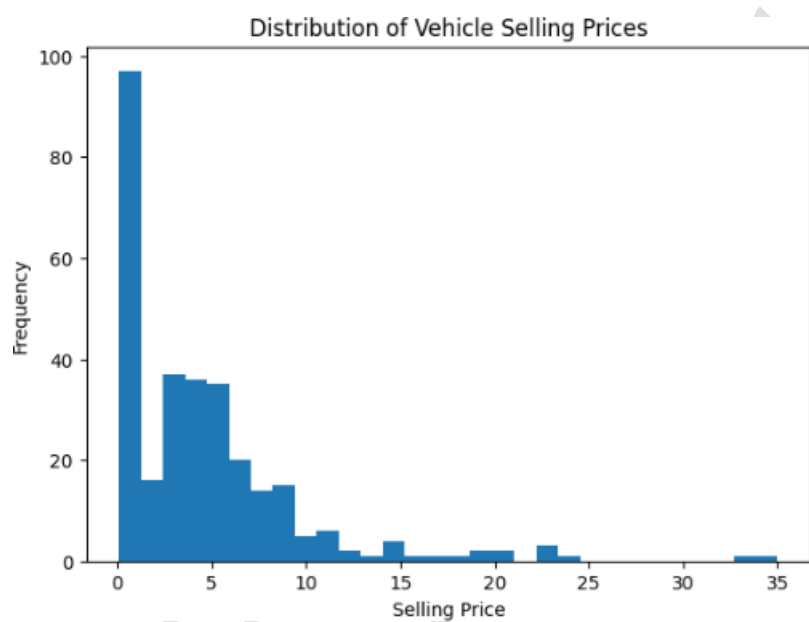
Fuel Type Aggregation

	selling_price	present_price	kms_driven
fuel_type			
CNG	3.100000	6.415000	42749.000000
Diesel	10.278500	15.814500	50369.916667
Petrol	3.264184	5.583556	33528.937238

Correlation Matrix

	year	selling_price	present_price	kms_driven	owner
year	1.000000	0.236141	-0.047584	-0.524342	-0.182104
selling_price	0.236141	1.000000	0.878983	0.029187	-0.088344
present_price	-0.047584	0.878983	1.000000	0.203647	0.008057
kms_driven	-0.524342	0.029187	0.203647	1.000000	0.089216
owner	-0.182104	-0.088344	0.008057	0.089216	1.000000





Fuel Type Counts

fuel_type

Petrol 239

Diesel 60

CNG 2

Name: count, dtype: int64

Unique Car Names

['ritz' 'sx4' 'ciaz' 'wagon r' 'swift' 'vitara brezza' 's cross'

'alto 800' 'ertiga' 'dzire' 'alto k10' 'ignis' '800' 'baleno' 'omni'

'fortuner' 'innova' 'corolla altis' 'etios cross' 'etios g' 'etios liva'

'corolla' 'etios gd' 'camry' 'land cruiser' 'Royal Enfield Thunder 500'

'UM Renegade Mojave' 'KTM RC200' 'Bajaj Dominar 400'

'Royal Enfield Classic 350' 'KTM RC390' 'Hyosung GT250R'

'Royal Enfield Thunder 350' 'KTM 390 Duke ' 'Mahindra Mojo XT300'

'Bajaj Pulsar RS200' 'Royal Enfield Bullet 350'

'Royal Enfield Classic 500' 'Bajaj Avenger 220' 'Bajaj Avenger 150'

'Honda CB Hornet 160R' 'Yamaha FZ S V 2.0' 'Yamaha FZ 16'

'TVS Apache RTR 160' 'Bajaj Pulsar 150' 'Honda CBR 150' 'Hero Extreme'

'Bajaj Avenger 220 dtsi' 'Bajaj Avenger 150 street' 'Yamaha FZ v 2.0'

'Bajaj Pulsar NS 200' 'Bajaj Pulsar 220 F' 'TVS Apache RTR 180'

'Hero Passion X pro' 'Bajaj Pulsar NS 200' 'Yamaha Fazer '

'Honda Activa 4G' 'TVS Sport ' 'Honda Dream Yuga '

'Bajaj Avenger Street 220' 'Hero Splender iSmart' 'Activa 3g'

'Hero Passion Pro' 'Honda CB Trigger' 'Yamaha FZ S '

'Bajaj Pulsar 135 LS' 'Activa 4g' 'Honda CB Unicorn'

'Hero Honda CBZ extreme' 'Honda Karizma' 'Honda Activa 125' 'TVS Jupyter'

'Hero Honda Passion Pro' 'Hero Splender Plus' 'Honda CB Shine'

'Bajaj Discover 100' 'Suzuki Access 125' 'TVS Wego' 'Honda CB twister'

'Hero Glamour' 'Hero Super Splendor' 'Bajaj Discover 125' 'Hero Hunk'

'Hero Ignitor Disc' 'Hero CBZ Xtreme' 'Bajaj ct 100' 'i20' 'grand i10'

'i10' 'eon' 'xcen' 'elantra' 'creta' 'verna' 'city' 'brio' 'amaze'

'jazz'

Unique Fuel Types

['Petrol' 'Diesel' 'CNG']

Unique Seller Types

['Dealer' 'Individual']

Unique Transmissions

['Manual' 'Automatic']

Top 10 Car Names by Number of Vehicles

car_name

city 26

corolla altis 16

verna 14

fortuner 11

brio 10

ciaz 9

i20 9

innova 9

grand i10 8

Royal Enfield Classic 350 7

Name: count, dtype: int64

Number of Vehicles by Year

year

2003 2

2004 1

2005 4

2006 4

2007 2

2008 7

2009 6

2010 15

2011 19

2012 23

2013 33

2014 38

2015 61

2016 50

2017 35

2018 1

Name: count, dtype: int64

Top 10 Car Names by Total Selling Price

car_name

fortuner 205.54

city 192.90

innova 115.00

corolla altis 114.93

verna 85.51

ciaz 67.25

brio	47.45
i20	42.90
jazz	40.80
ertiga	40.65

Name: selling_price, dtype: float64

Most Common Fuel Types

fuel_type

Petrol 239

Diesel 60

CNG 2

Name: count, dtype: int64

CSV, Excel and JSON files exported successfully

=====

EXPERIMENT COMPLETED SUCCESSFULLY

=====

Dataset Shape : (301, 9)

Missing Values : 0

Total Vehicles : 301

Total Selling Price : 1403.0500000000002

Average Selling Price : 4.661295681063123

Number of Car Names : 98

Number of Fuel Types : 3

Number of Seller Types : 2

Number of Transmissions : 2

Correlation Matrix Generated

Result:

Ex.no:5a	DATA VISUALIZATION USING MATPLOTLIB – SALES DATA

Aim:

To visualize dataset features and distributions using line charts, bar charts, pie charts, histograms, scatter plots, and box plots using Matplotlib.

Algorithm:

ALGORITHM 1: LINE CHART

Objective: To visualize the monthly sales trend.

Steps

1. Load the sales dataset.
2. Select the Month and Sales columns.
3. Plot the monthly sales using a line chart.
4. Add title and axis labels.
5. Display the graph.

Interpretation

The line chart shows how sales vary across different months and helps identify the sales trend.

Program 1: Line Chart

```
plt.plot(months, sales)
plt.title("Monthly Sales")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.show()
```

ALGORITHM 2: BAR CHART

Objective: To compare sales across different months.

Steps

1. Select the Month and Sales columns.
2. Create a bar chart using the monthly sales values.
3. Add title and axis labels.
4. Display the chart.

Interpretation

The bar chart helps compare the sales values across different months.

Program 2: Bar Chart

```
plt.bar(months, sales)
plt.title("Monthly Sales")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.show()
```

ALGORITHM 3: PIE CHART

Objective: To visualize the distribution of sales across different months.

Steps

1. Select the sales values.
2. Create a pie chart using the sales values.
3. Display percentage labels.
4. Add a suitable title.
5. Display the chart.

Interpretation

The pie chart shows the percentage contribution of each month to the total sales.

Program 3: Pie Chart

```
plt.pie(sales, labels=months, autopct="%1.1f%%")
plt.title("Sales Distribution")
plt.show()
```

ALGORITHM 4: HISTOGRAM

Objective: To visualize the distribution of sales values.

Steps

1. Select the Sales column.
2. Divide the sales values into bins.
3. Create a histogram.
4. Add title and axis labels.
5. Display the histogram.

Interpretation

The histogram shows the frequency distribution of the monthly sales values.

Program 4: Histogram

```
plt.hist(sales, bins=5)
plt.title("Sales Distribution")
```

```
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.show()
```

ALGORITHM 5: SCATTER PLOT

Objective: To visualize the sales values for different months.

Steps

1. Select Month as the X-axis variable.
2. Select Sales as the Y-axis variable.
3. Create a scatter plot.
4. Add title and axis labels.
5. Display the graph.

Interpretation

The scatter plot shows the variation in sales across different months.

Program 5: Scatter Plot

```
plt.scatter(months, sales)
plt.title("Monthly Sales")
plt.xlabel("Months")
plt.ylabel("Sales")
plt.show()
```

ALGORITHM 6: BOX PLOT

Objective: To visualize the distribution of monthly sales.

Steps

1. Select the Sales column.
2. Create a box plot.
3. Add a suitable title and axis label.
4. Display the plot.

Interpretation

The box plot shows the distribution of sales values and helps identify possible outliers.

Program 6: Box Plot

```
plt.boxplot(sales)
plt.title("Sales Box Plot")
plt.ylabel("Sales")
plt.show()
```

ALGORITHM 7: MULTIPLE LINE CHARTS

Objective: To compare sales and profit across different months.

Steps

1. Store the profit values.
2. Plot the sales values using a line chart.
3. Plot the profit values using another line chart.
4. Add title, axis labels and legend.
5. Display the graph.

Interpretation

The multiple line chart compares the sales and profit values across different months.

Program 7: Multiple Line Charts

```
profit = [20, 30, 40, 35, 50, 60]
```

```
plt.plot(months, sales, label="Sales")
```

```
plt.plot(months, profit, label="Profit")
```

```
plt.title("Sales and Profit")
```

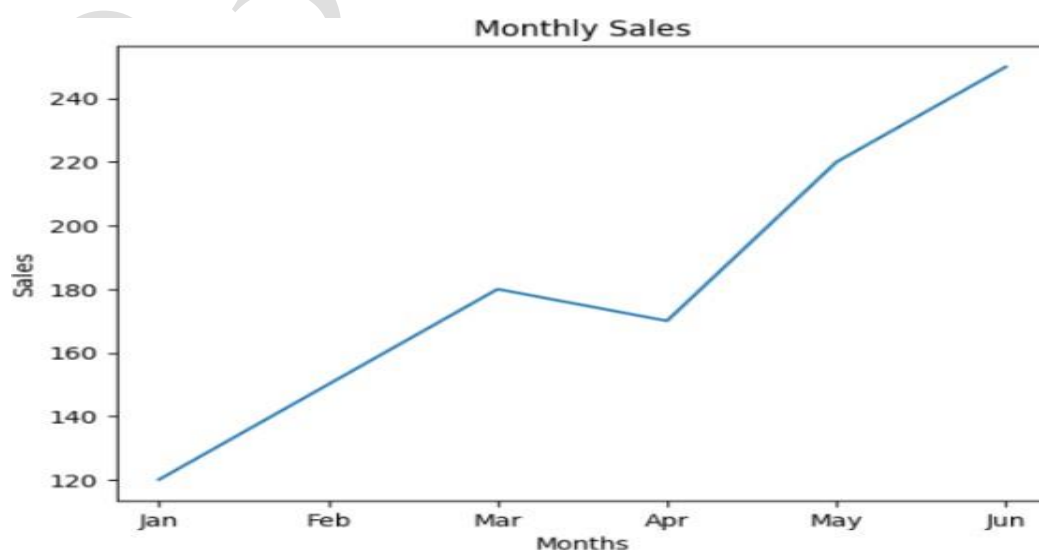
```
plt.xlabel("Months")
```

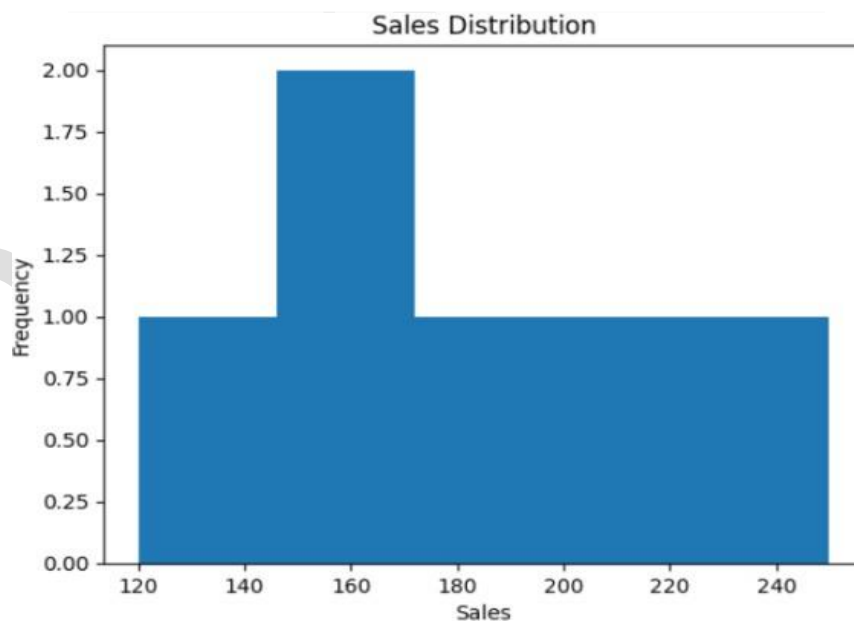
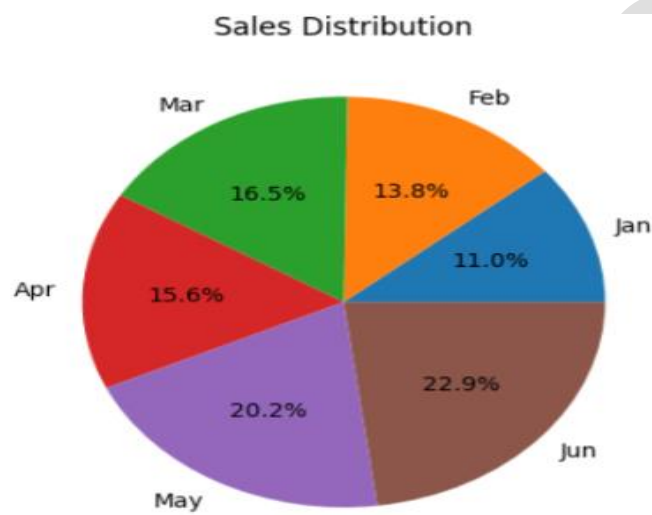
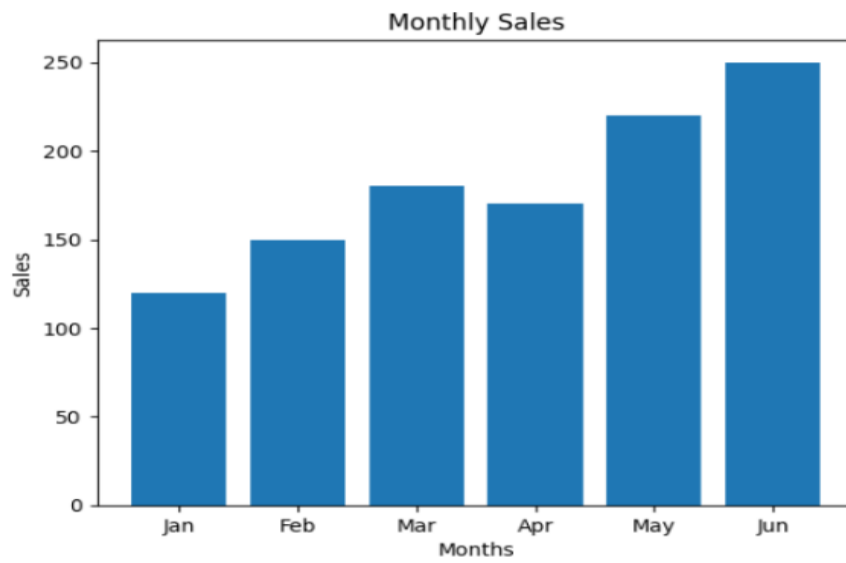
```
plt.ylabel("Value")
```

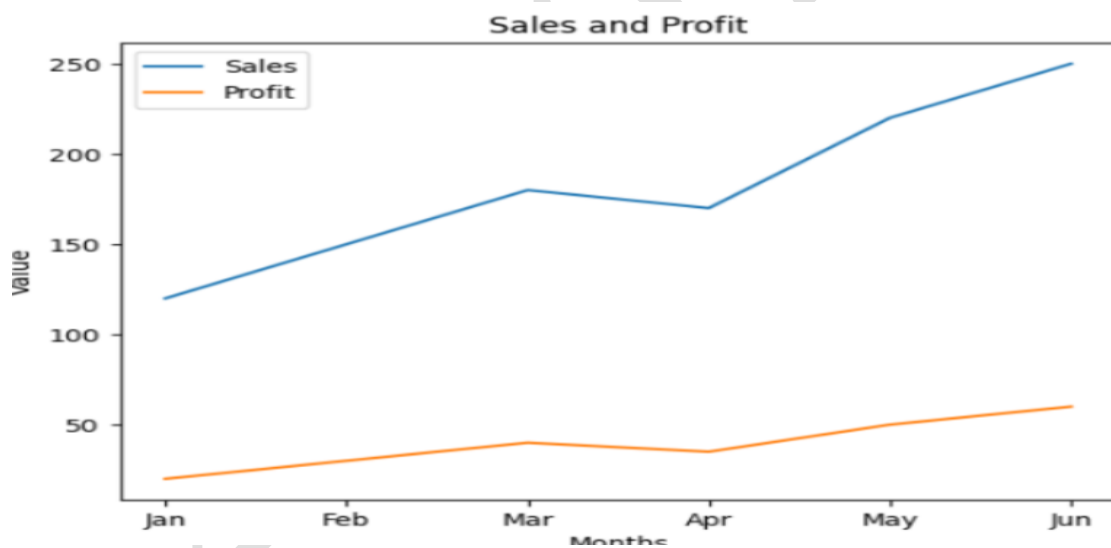
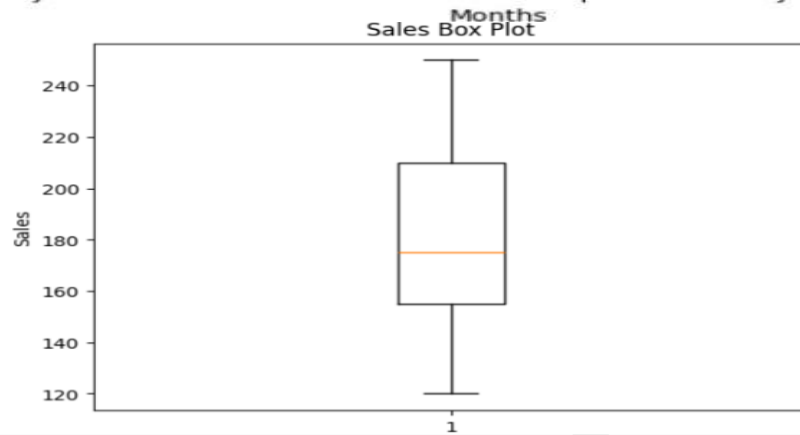
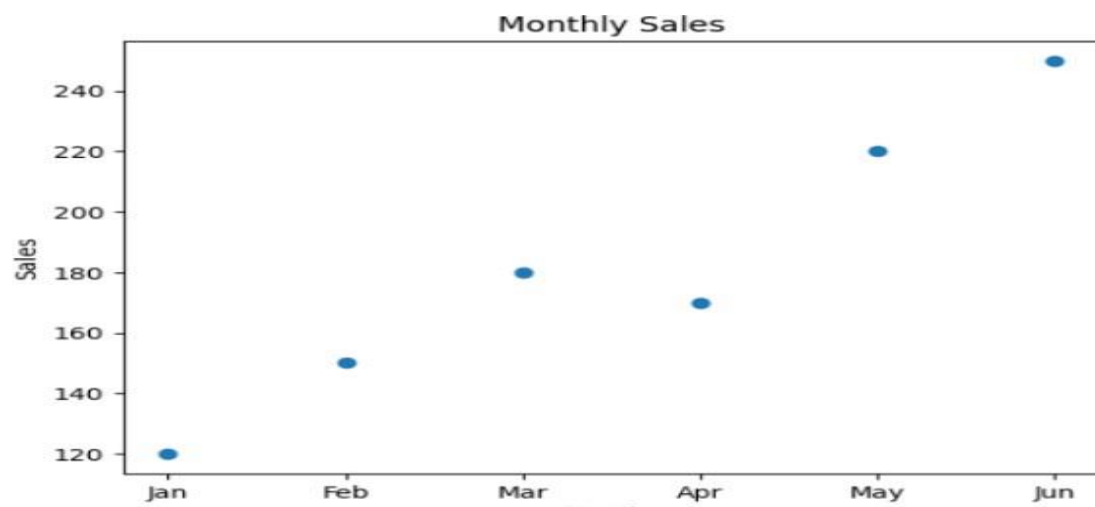
```
plt.legend()
```

```
plt.show()
```

Output:







Result:

Ex.no:5b

DATA VISUALIZATION USING MATPLOTLIB – SALES DATA

AIM

To visualize and analyze the Vehicle Analysis dataset using Line Chart, Bar Chart, and Pie Chart with Matplotlib to understand attack trends, regional comparisons, and attack distribution.

ALGORITHM

Step 1: Import Pandas and Matplotlib libraries.

Step 2: Load the Vehicle Analysis dataset into a DataFrame

Step 3: Group the vehicle market according to year and calculate the number of vehicles for each year.

Step 4: Plot the yearly sales count using a Line Chart to observe the trend of vehicle market over time.

Step 5: Group the dataset according to region and calculate the number of fuel in each region.

Step 6: Plot the regional sales counts using a Bar Chart to compare vehicle market sales across different regions.

Step 7: Select the major regions and calculate their sales proportions

Step 8: Plot the regional distribution using a Pie Chart.

Step 9: Add suitable titles, axis labels, legends, and display the visualizations.

CODE

```
import pandas as pd
import matplotlib.pyplot as plt
import kagglehub
import os

# Load Vehicle Dataset from Kaggle
path = kagglehub.dataset_download(
    "nehalbirla/vehicle-dataset-from-cardekho"
)

print("Dataset Path:")
print(path)

# Find CSV file automatically
csv_files = [f for f in os.listdir(path) if f.endswith(".csv")]
csv_file = csv_files[0]

# Load Dataset
df = pd.read_csv(
```

```

os.path.join(path, csv_file),
encoding="latin1",
low_memory=False
)
# Clean column names
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")
print("\nDataset Loaded Successfully")
print("\nColumn Names:")
print(df.columns.tolist())
# -----
# 1. Line Chart – Vehicles by Year
# -----
yearly_vehicles = df.groupby("year").size()
plt.figure(figsize=(10, 5))
plt.plot(
    yearly_vehicles.index,
    yearly_vehicles.values
)
plt.title("Vehicles by Year")
plt.xlabel("Year")
plt.ylabel("Number of Vehicles")
plt.grid(True)
plt.show()
# -----
# 2. Bar Chart – Vehicles by Fuel Type
# -----
fuel_vehicles = df["fuel_type"].value_counts()
plt.figure(figsize=(10, 6))
plt.bar(
    fuel_vehicles.index,
    fuel_vehicles.values
)
plt.title("Vehicles by Fuel Type")
plt.xlabel("Fuel Type")

```

```

plt.ylabel("Number of Vehicles")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

# -----

# 3. Pie Chart – Fuel Type Distribution
# -----

top_fuels = fuel_vehicles.head(6)

plt.figure(figsize=(8, 8))

plt.pie(
top_fuels.values,

    labels=top_fuels.index,
    autopct="%1.1f%"
)

plt.title("Fuel Type Distribution of Vehicles")
plt.show()

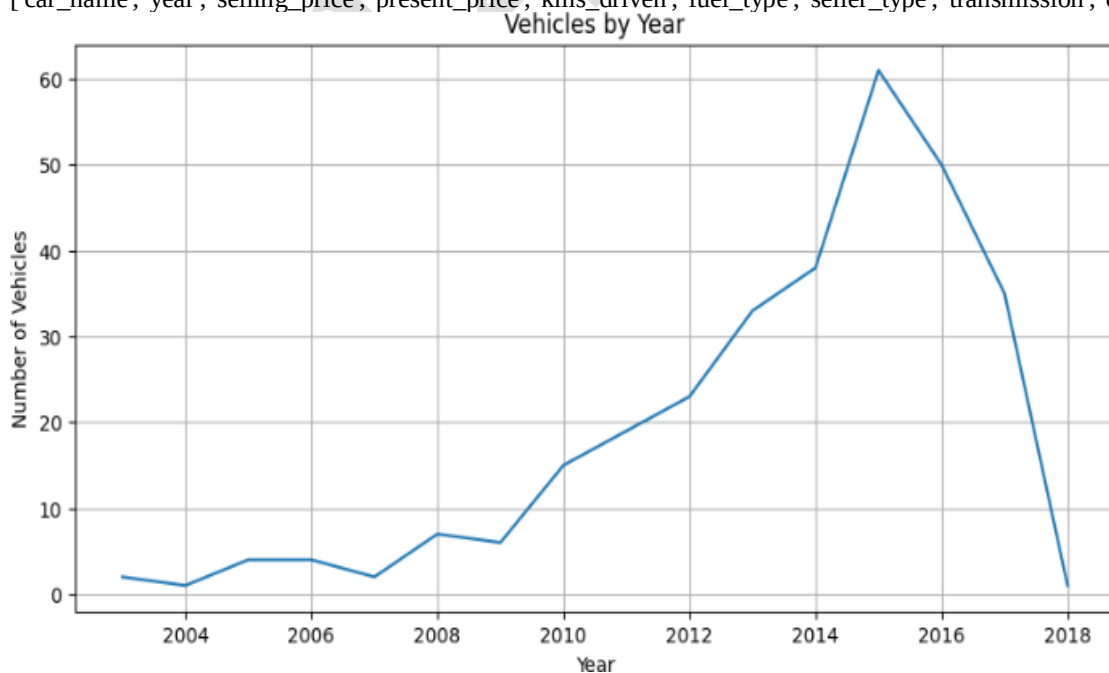
```

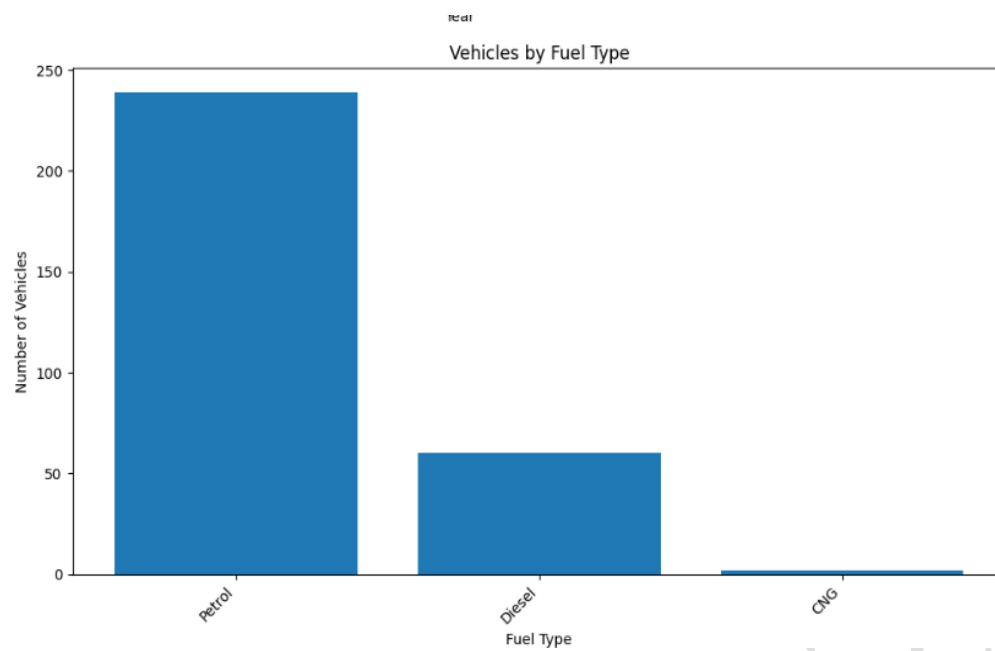
OUTPUT

Dataset Loaded Successfully

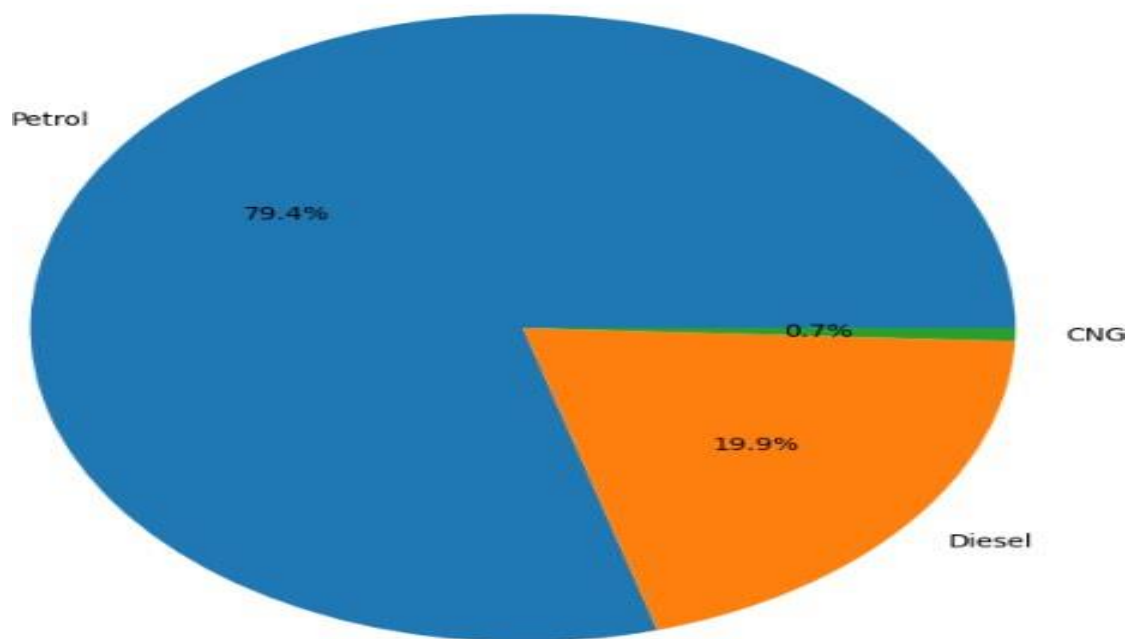
Column Names:

['car_name', 'year', 'selling_price', 'present_price', 'kms_driven', 'fuel_type', 'seller_type', 'transmission', 'owner']





Fuel Type Distribution of Vehicles



RESULT

DATA VISUALIZATION USING POWER BI**Aim:**

To Load the data into PowerBI and transform the data and visualize it to get insights

Algorithm:**Algorithm: Sales Dashboard Calculation****Step 1: Input Data Loading**

- Read the dataset from datas.csv containing columns Product, Region, and Sales.
- Dataset rows:
 - Laptop, North, 50000
 - Mobile, South, 35000
 - Printer, East, 25000
 - Laptop, West, 45000
 - Mobile, North, 40000
 - Printer, South, 30000

Step 2: Region Filter Check

- Check slicer selection for Region.
- Default selected regions: East, North, South, West.
- Retain all rows matching the selected regions in Filtered_Data.

Step 3: KPI Total Sales Calculation

- Sum all values in the Sales column from Filtered_Data.
- Total Sales = $50000 + 35000 + 25000 + 45000 + 40000 + 30000 = 225000$ (displayed as 225K).

Step 4: Bar Chart Calculation (Sum of Sales by Product)

- Group Filtered_Data by Product and sum Sales for each product:
 - Laptop Sales = $50000 \text{ (North)} + 45000 \text{ (West)} = 95000$
 - Mobile Sales = $35000 \text{ (South)} + 40000 \text{ (North)} = 75000$
 - Printer Sales = $25000 \text{ (East)} + 30000 \text{ (South)} = 55000$
- Sort products in descending order by total sales: Laptop, Mobile, Printer.

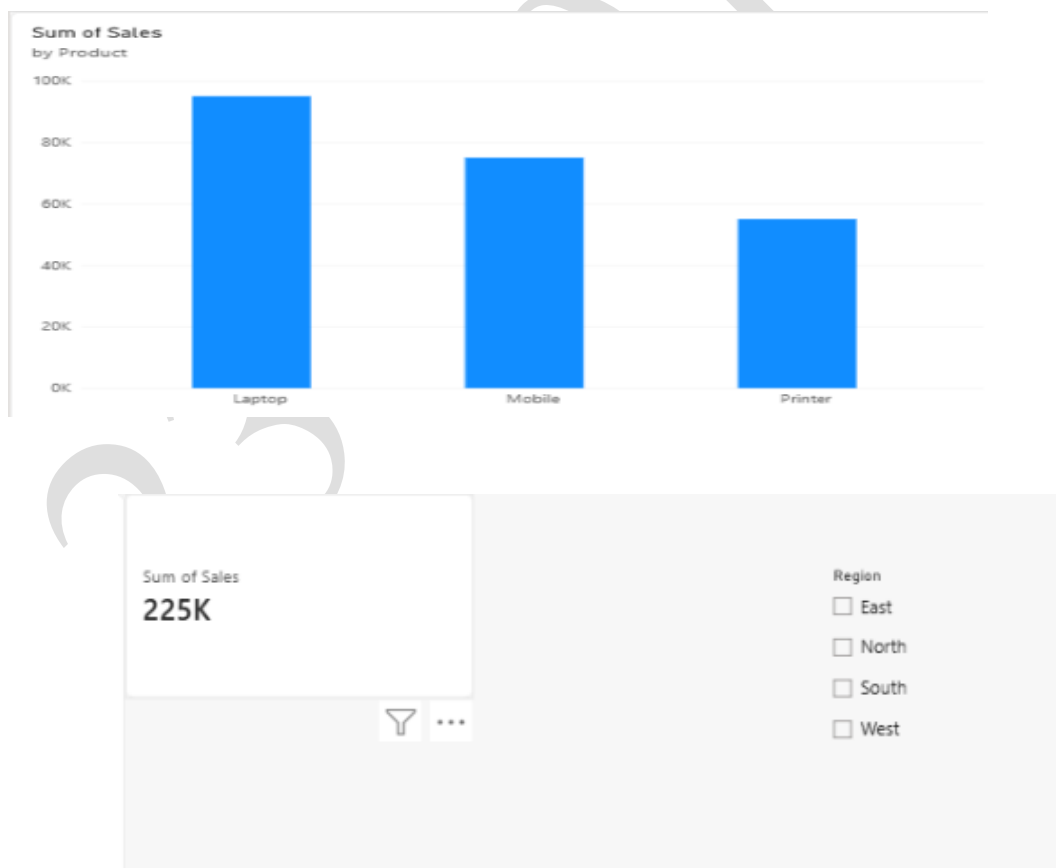
Step 5: Donut Chart Calculation (Sum of Sales by Region)

- North Sales = $50000 + 40000 = 90000$
- South Sales = $35000 + 30000 = 65000$
- West Sales = 45000
- East Sales = 25000
- Calculate percentage share for each region relative to Total Sales (225000):
 - North Share = $(90000 / 225000) * 100 = 40.00\%$
 - South Share = $(65000 / 225000) * 100 = 28.89\%$
 - West Share = $(45000 / 225000) * 100 = 20.00\%$
 - East Share = $(25000 / 225000) * 100 = 11.11\%$

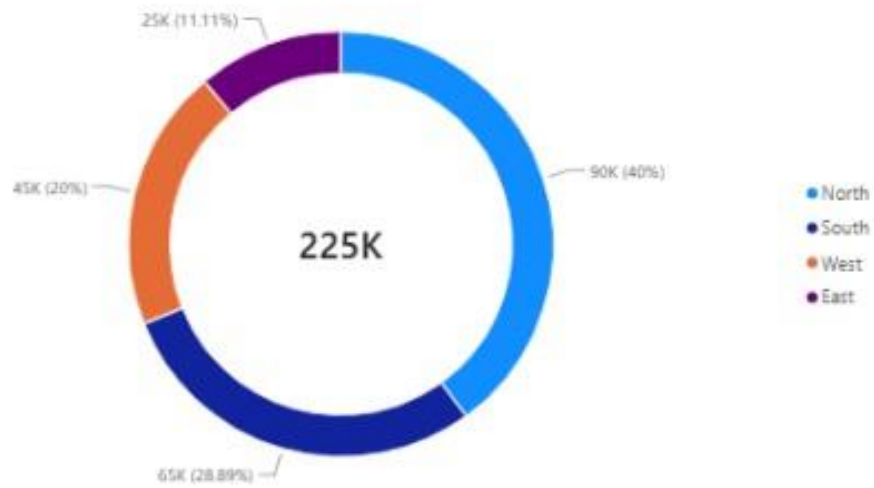
Step 6: Dashboard Render

- Render KPI card with formatted total value 225K.
- Render vertical bar chart with Laptop at 95K, Mobile at 75K, Printer at 55K.
- Render donut chart with region segments North (90K, 40%), South (65K, 28.89%), West (45K, 20%), East (25K, 11.11%), and center value 225K.

Output:



Sum of Sales
by Region



Result:

Ex.no:6b	DATA VISUALIZATION USING POWER BI

AIM

To visualize and analyze the Vehicle Analysis dataset using Line Chart, Bar Chart, and Pie Chart with Matplotlib to understand attack trends, regional comparisons, and attack distribution.

ALGORITHM

Step 1: Import Pandas and Matplotlib libraries.

Step 2: Load the Vehicle Analysis dataset into a DataFrame.

Step 3: Group the fuel types according to year and calculate the number of vehicle for each year.

Step 4: Plot the yearly vehicle count using a Line Chart to observe the trend of fuel type over time.

Step 5: Group the dataset according to region and calculate the number of vehicles in each region.

Step 6: Plot the regional fuel type counts using a Bar Chart to compare vehicle market across different regions.

Step 7: Select the major regions and calculate their fuel proportions.

Step 8: Plot the regional distribution using a Pie Chart.

Step 9: Add suitable titles, axis labels, legends, and display the visualizations.

CODE

```
import pandas as pd
import matplotlib.pyplot as plt
import kagglehub
import os

# Load Vehicle Dataset from Kaggle
path = kagglehub.dataset_download(
    "nehalbirla/vehicle-dataset-from-cardekho"
)

print("Dataset Path:")
```

```
print(path)

# Find CSV file automatically

csv_files = [f for f in os.listdir(path) if f.endswith(".csv")]

csv_file = csv_files[0]

# Load Dataset

df = pd.read_csv(

    os.path.join(path, csv_file),

    encoding="latin1",

    low_memory=False

)

# Clean column names

df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")

print("\nDataset Loaded Successfully")

# -----

# 1. Line Chart – Vehicles by Year

# -----

yearly_vehicles = df.groupby("year").size()

plt.figure(figsize=(10, 5))

plt.plot(

    yearly_vehicles.index,

    yearly_vehicles.values

)

plt.title("Vehicles by Year")

plt.xlabel("Year")

plt.ylabel("Number of Vehicles")

plt.grid(True)
```

```
plt.show()

# -----

# 2. Bar Chart – Vehicles by Fuel Type

# -----

fuel_vehicles = df["fuel_type"].value_counts()

plt.figure(figsize=(12, 6))

plt.bar(

    fuel_vehicles.index,

    fuel_vehicles.values

)

plt.title("Vehicles by Fuel Type")

plt.xlabel("Fuel Type")

plt.ylabel("Number of Vehicles")

plt.xticks(rotation=45, ha="right")

plt.tight_layout()

plt.show()

# -----

# 3. Pie Chart – Fuel Type Distribution

# -----

top_fuels = fuel_vehicles.head(6)

plt.figure(figsize=(8, 8))

plt.pie(

    top_fuels.values,

    labels=top_fuels.index,

    autopct="%1.1f%%"

)
```

```
plt.title("Top 6 Fuel Types by Number of Vehicles")
```

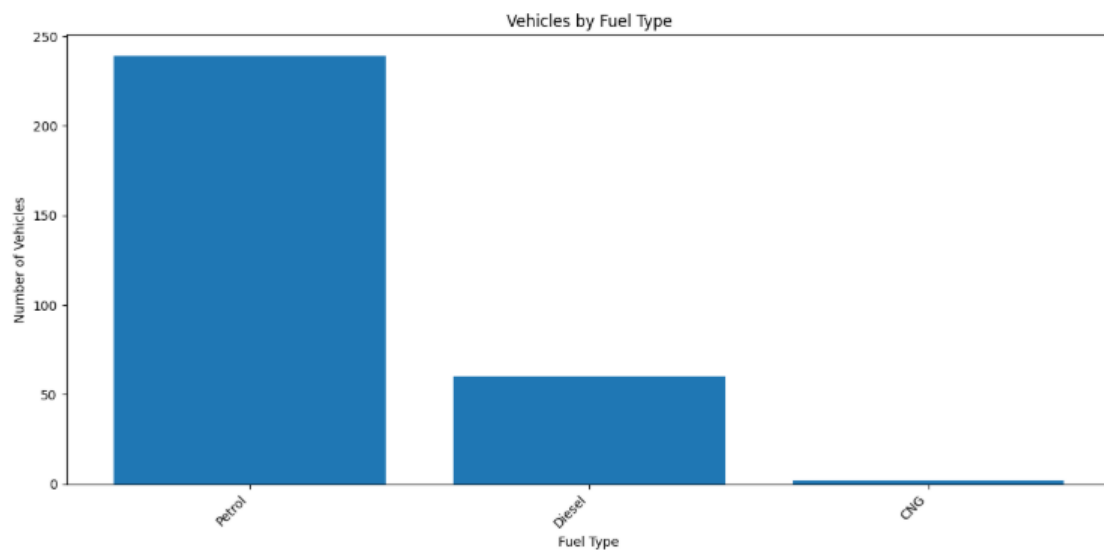
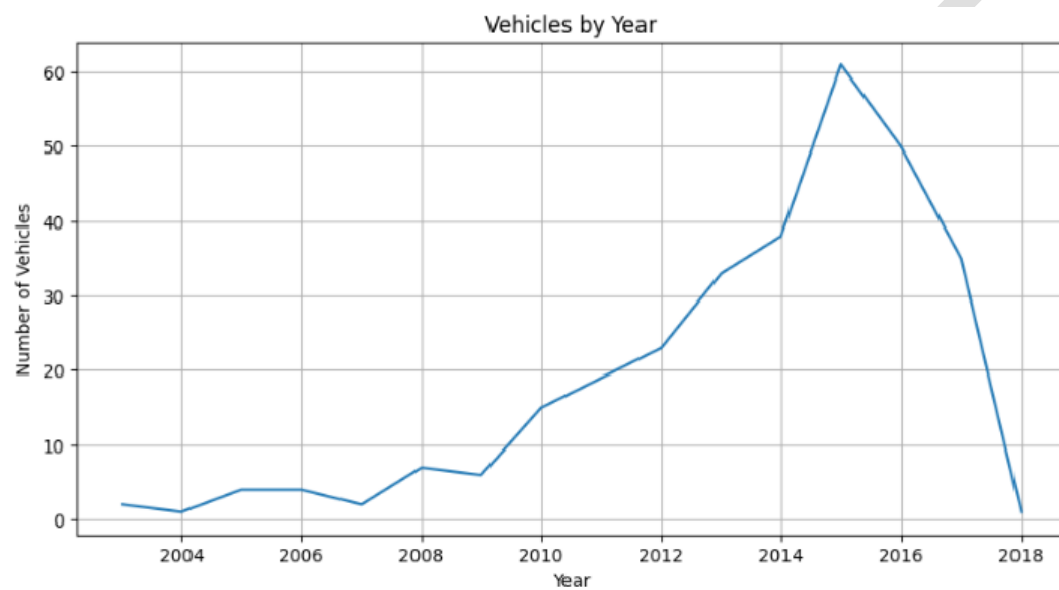
```
plt.show()
```

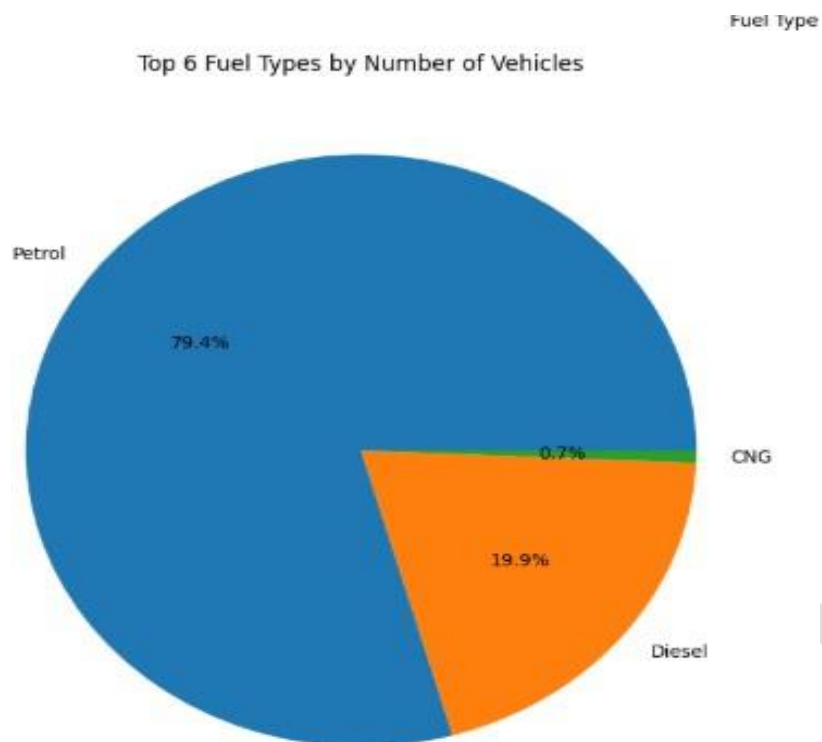
OUTPUT

Dataset Path:

/kaggle/input/vehicle-dataset-from-cardekho

Dataset Loaded Successfully





RESULT

Ex.no:6

DATA VISUALIZATION USING POWER BI (VEHICLE ANALYSIS)

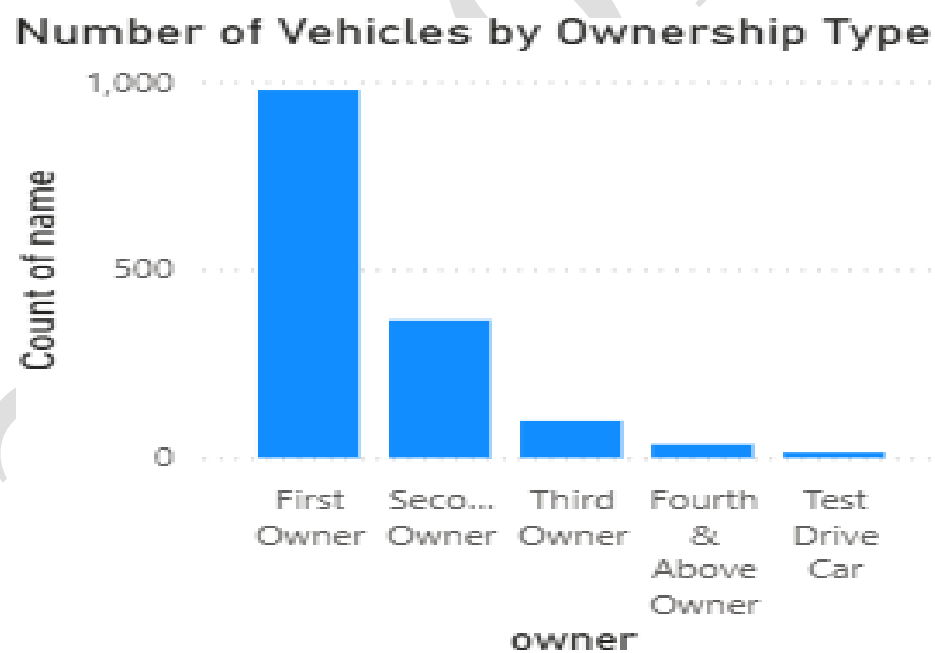
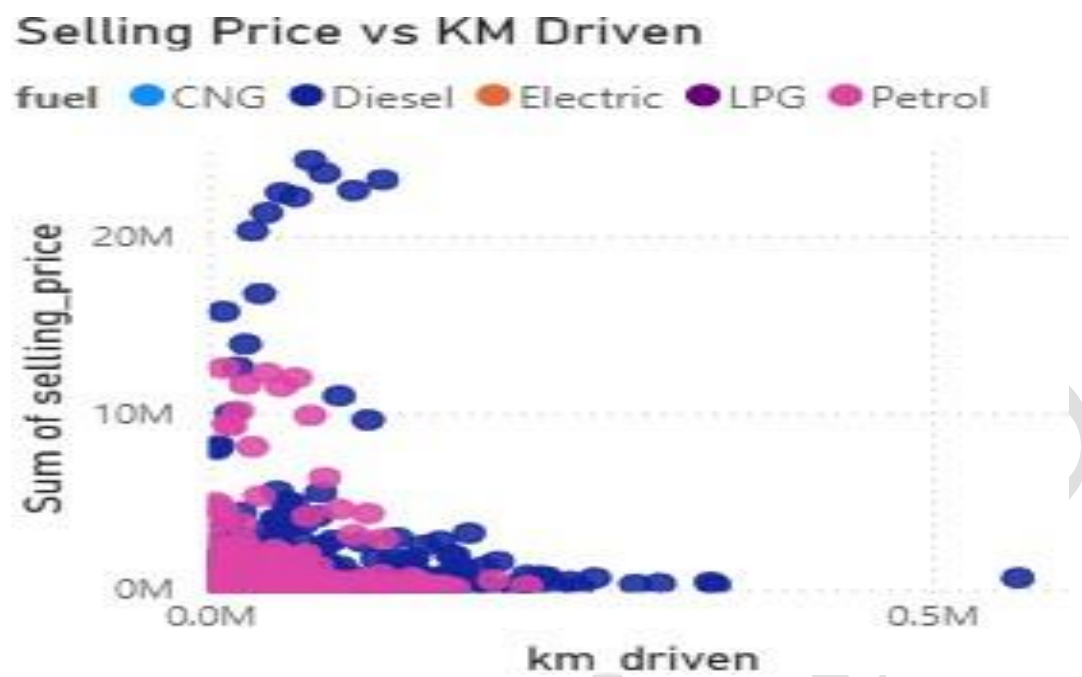
Aim:

To clean, preprocess, and transform the raw vehicle analysis dataset, load the clean data model into Power BI, and build an interactive visual dashboard to analyze vehicle prices, usage, fuel types, transmission types, seller types, and ownership.

Algorithm:

1. **Import Dataset:** Load the **vehicle analysis CSV dataset** into Power BI Desktop using **Get Data → Text/CSV**.
2. **Remove Duplicate Rows:** Identify and remove duplicate records from the dataset to ensure each vehicle entry is unique.
3. **Handle Missing Values:** Identify columns containing null/missing values and replace them appropriately using 0, median, or mode, depending on the column.
4. **Handle Data Type Errors:**
Replace invalid or non-numeric values such as "NA" or blank values in numerical columns with appropriate values such as 0.
5. **Format Data Types:**
Set numerical fields such as **Price, Mileage, Engine, Power, and Year** to the appropriate numeric data types.
Set categorical fields such as **Car Name, Fuel Type, Seller Type, Transmission, and Owner Type** as Text.
6. **Clean and Transform Data:**
Standardize categorical values and remove unnecessary columns that are not required for vehicle analysis.
7. **Load Clean Data:**
Apply the transformations using **Power Query → Close & Apply** to load the cleaned vehicle dataset into Power BI.
8. **Create Dashboard:**
Create an interactive **Vehicle Analysis Dashboard** by combining the required metrics and visualizations.

Output:



Average Selling Price by Year



Fuel Type

fuel

☐ CNG

☐ Diesel

☐ Electric

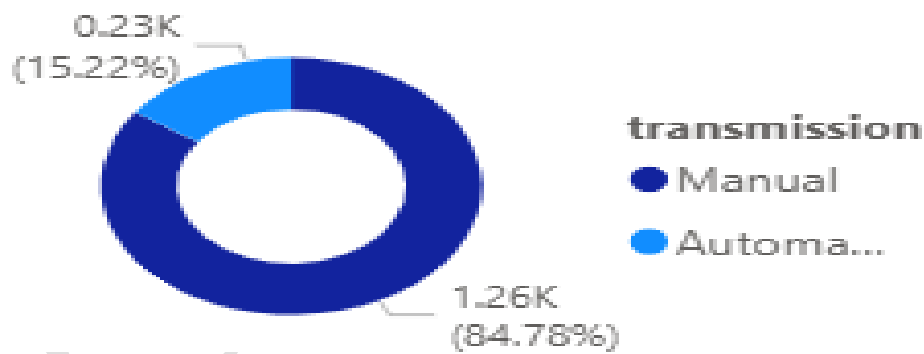
Transmission

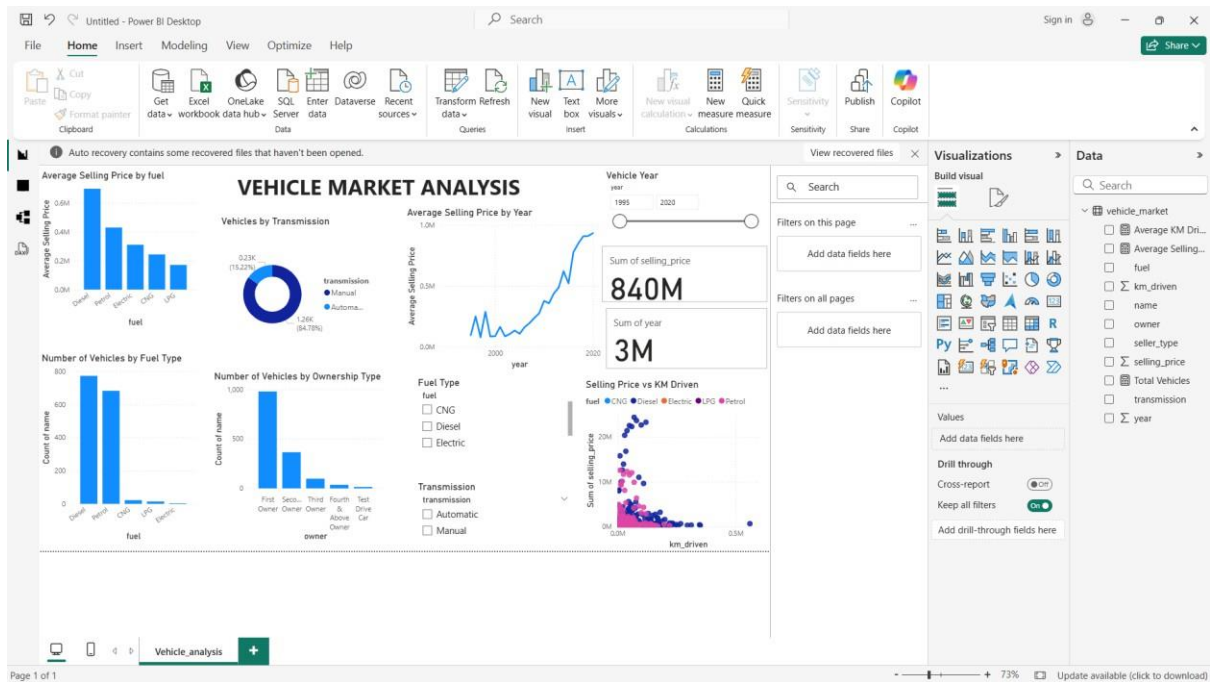
transmission

☐ Automatic

☐ Manual

Vehicles by Transmission





Result:

DATA VISUALIZATION USING TABLEAU

Aim:

To connect, analyze, visualize, and create interactive dashboards using Tableau for effective business intelligence and data analytics.

Algorithm:

Algorithm: Student Performance Visualization using Tableau

Step 1: Data Loading

- Connect the synthetic Student Performance dataset to Tableau.
- Verify the imported fields: Student, Department, and Marks.
- The dataset contains six students: Arun, Bala, Divya, Priya, Ravi, and Karthik.

Step 2: Student Marks Bar Chart

- Create a worksheet for student-wise marks.
- Place Student on Columns.
- Place Marks on Rows.
- Select the Bar Chart visualization.
- Enable data labels to display the marks of each student.

Step 3: Department-wise Marks Visualization

- Create a worksheet using Department and Marks.
- Aggregate Marks using SUM.
- Visualize the contribution of each department using a Pie Chart.
- Display department names and corresponding marks as labels.

Step 4: Student Performance Trend

- Create a Line Chart using Student and Marks.
- Place Student on Columns and Marks on Rows.
- Enable labels to display individual marks.
- Use the chart to compare the performance of students.

Step 5: Marks Percentage

- Create a calculated field named Percentage.

- Use the formula:
[Marks]
- Display Student, Marks, and Percentage in a worksheet.

Step 6: Department Filter

- Add Department to the Filters shelf.
- Select a department such as AI&DS.
- Observe the updated student marks based on the selected department.

Step 7: Minimum Marks Parameter

- Create a parameter named Minimum Marks.
- Set the required minimum marks value.
- Apply the parameter to display only students whose marks meet or exceed the selected value.
- For example, when Minimum Marks = 85, display students scoring 85 or above.

Step 8: Dashboard Creation

- Create a new Tableau Dashboard.
- Add the Student Marks, Department Contribution, Marks Percentage, Student Performance Trend, and Minimum Marks visualizations.
- Arrange the worksheets clearly.
- Add the Department filter and Minimum Marks parameter.
- Format the dashboard with suitable titles and labels.

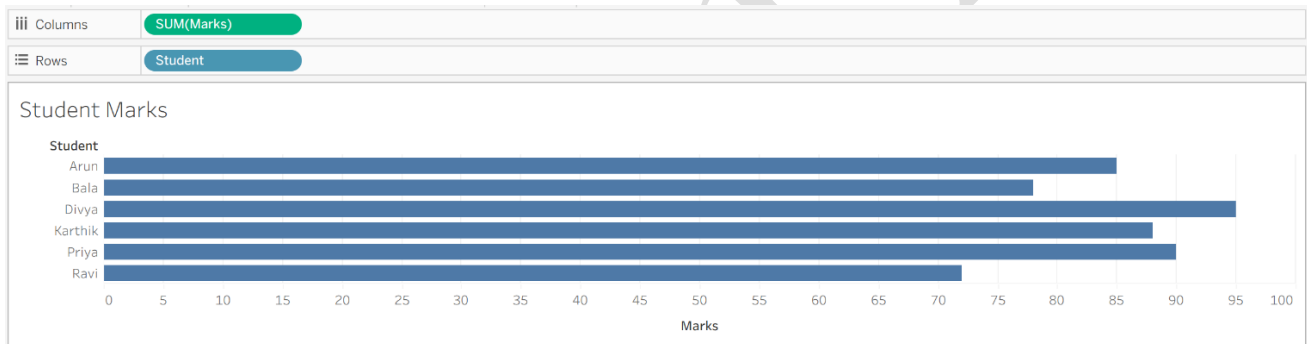
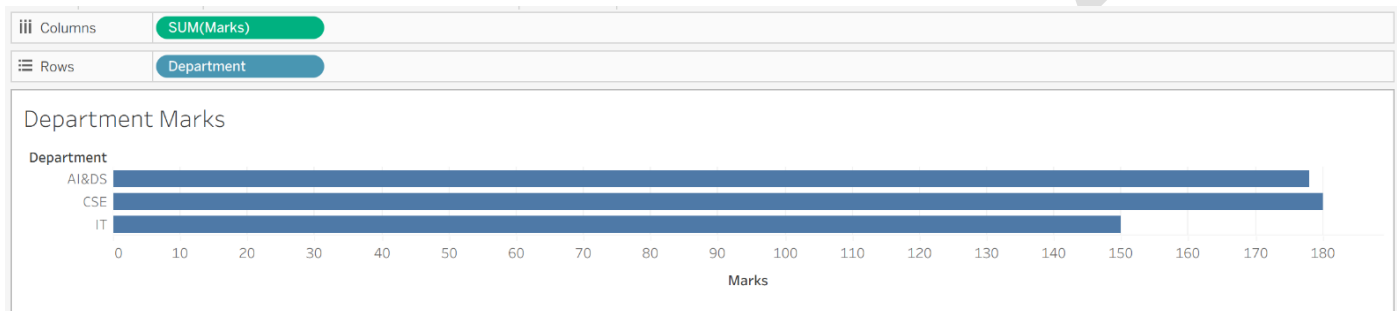
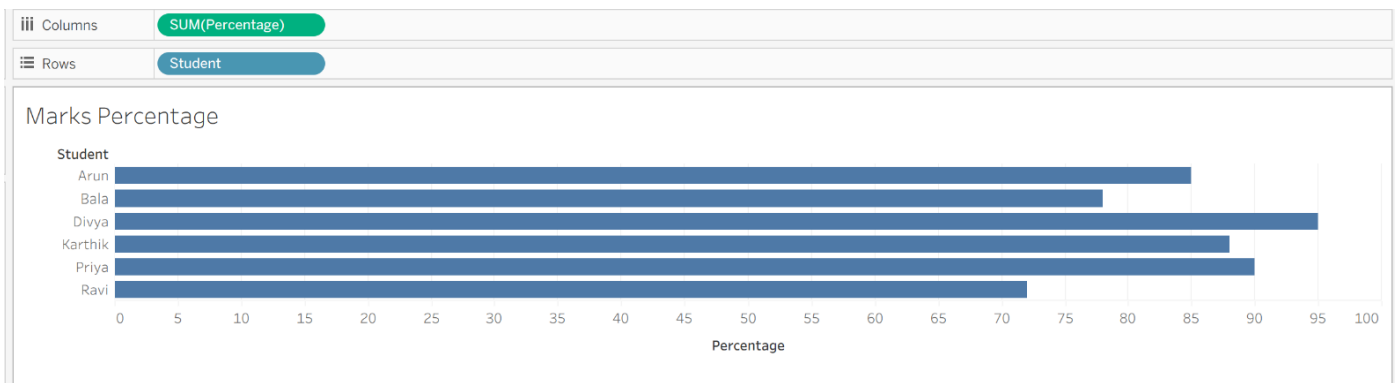
Step 9: Story Creation

- Create a Tableau Story using the completed dashboard.
- Add story points for:
 1. Overall Student Performance
 2. Department-wise Analysis
 3. Top Performing Students
 4. Final Performance Analysis
- Present the visual insights using the story.

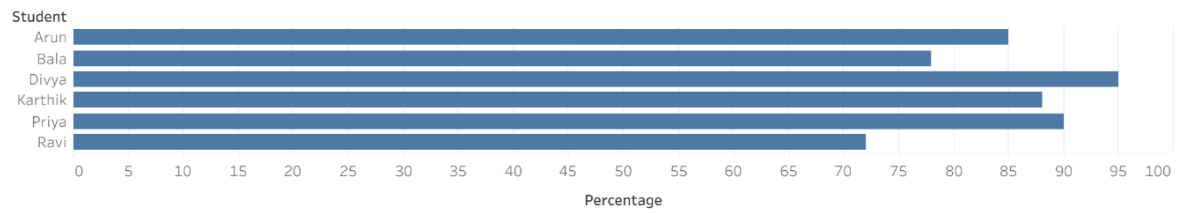
Step 10: Analysis

- Analyze the charts to identify the highest and lowest performing students.
- Compare marks among departments.
- Observe how the Department filter and Minimum Marks parameter dynamically change the visualizations.
- Use the dashboard and story to summarize the overall student performance.

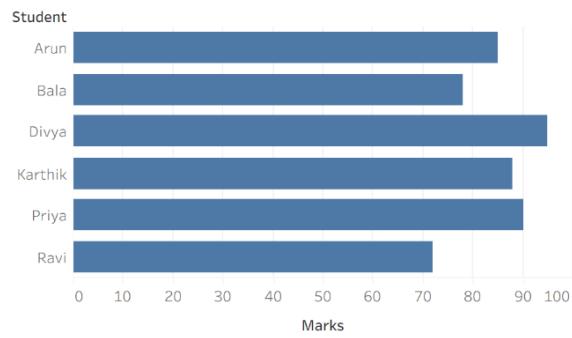
Output:



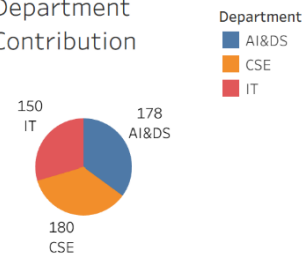
Marks Percentage



Student Marks



Department Contribution

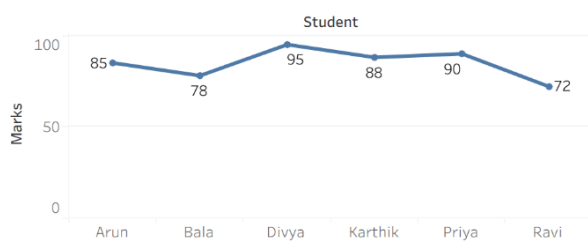


Minimum Marks

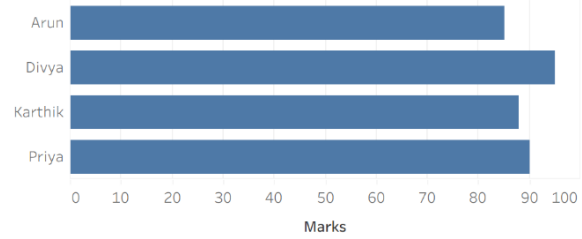


Minimum Marks Parameter

Student Performance Trend



Student



Result:

Ex.no:7b

DATA VISUALIZATION USING TABLEAU- VEHICLE ANALYSIS DATASET

Aim

To connect, analyze, visualize, and create interactive dashboards using Tableau with the **Vehicle Analysis Dataset** for effective analysis of vehicle prices, usage, fuel types, transmission types, seller types, and ownership.

Software Requirements

- Tableau Desktop / Tableau Public
- Microsoft Excel
- CSV Dataset
- Windows Operating System

Theory

Tableau is a powerful Business Intelligence (BI) and Data Visualization tool used to create interactive dashboards and reports.

The **Vehicle Analysis Dataset (VAD)** contains information about used vehicles, including:

- Car Name
- Manufacturing Year
- Selling Price
- Present Price
- Kilometers Driven
- Fuel Type
- Seller Type
- Transmission
- Number of Previous Owners

Tableau allows users to analyze vehicle prices, usage patterns, fuel preferences, transmission types, and ownership characteristics through interactive visualizations.

Tableau Architecture

Vehicle Analysis Dataset



Data Connection



Worksheets



Visualizations



Dashboard



Story

Learning Outcomes

After completing this experiment, students will be able to:

- ✓ Connect the Vehicle Analysis Dataset to Tableau
- ✓ Create worksheets using dimensions and measures
- ✓ Generate different charts and visualizations
- ✓ Analyze vehicle prices
- ✓ Analyze vehicle usage based on kilometers driven
- ✓ Apply filters and parameters
- ✓ Create calculated fields
- ✓ Build interactive dashboards
- ✓ Create stories for vehicle data analysis

Sample Dataset

Vehicle Analysis Dataset

Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Transmission
ritz	2014	3.35	5.59	27000	Petrol	Manual
sx4	2013	4.75	9.54	43000	Diesel	Manual
ciaz	2017	7.25	9.85	6900	Petrol	Manual
wagon r	2011	2.85	4.15	5200	Petrol	Manual
swift	2014	4.60	6.87	42450	Diesel	Manual

The actual VAD contains a much larger number of records and columns.

Algorithm 1 – Connecting Data Source

Step 1

Open **Tableau Desktop**.

Step 2

Select **Connect**.

Step 3

Choose **Text File**.

Step 4

Browse and select the Vehicle Analysis Dataset.

Step 5

Open the dataset.

Step 6

Verify the imported fields such as:

- Year
- Car_Name
- Selling_Price
- Present_Price
- Kms_Driven
- Fuel_Type
- Seller_Type
- Transmission
- Owner

Procedure

Tableau



Connect



Excel / Text File



Vehicle Analysis Dataset

↓

Open

↓

Verify Data

Output

Vehicle Analysis Dataset loaded into Tableau successfully.

Algorithm 2 – Creating Worksheet

Step 1

Open **Sheet 1**.

Step 2

Drag **Fuel_Type** to Columns.

Step 3

Drag **Selling_Price** to Rows.

Step 4

Change Selling_Price aggregation to **Average**.

Step 5

Select an appropriate visualization.

Output

Worksheet created successfully.

Interpretation

The worksheet displays the average selling price of vehicles according to their fuel type.

Algorithm 3 – Bar Chart Creation

Objective

To compare the average selling price of vehicle based on fuel type.

Step 1

Drag **Fuel_Type** → Columns.

Step 2

Drag **Selling_Price** → Rows.

Step 3

Select **Bar Chart** from Show Me.

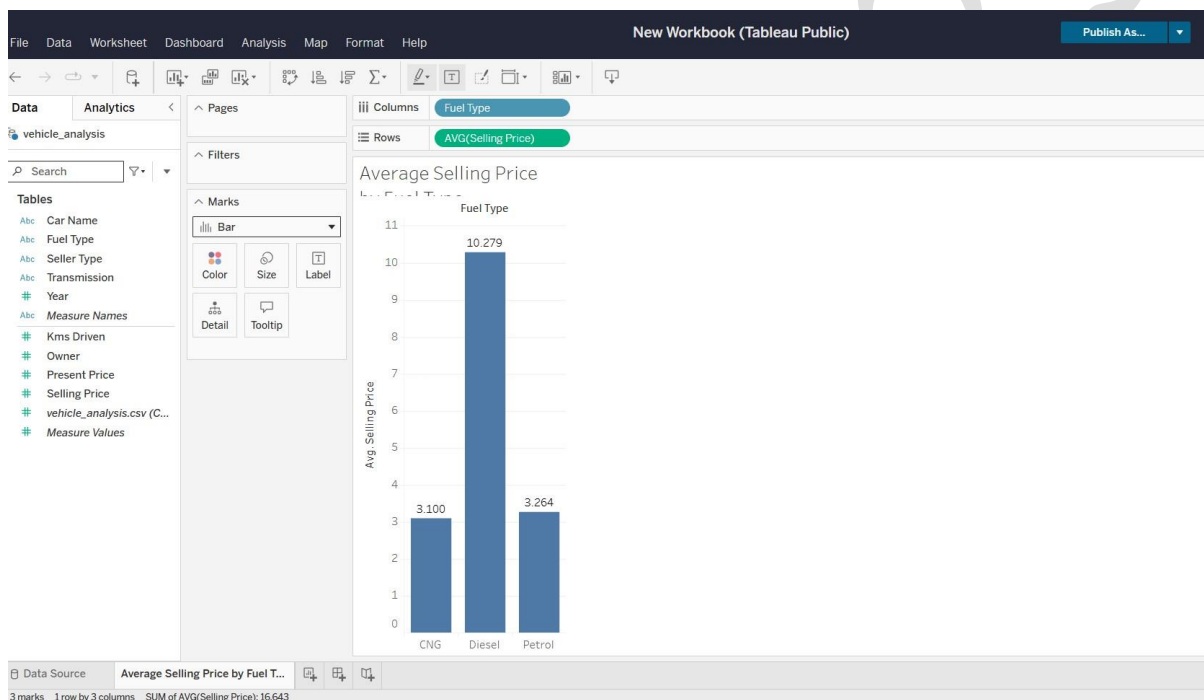
Step 4

Change Selling_Price to **Average**.

Step 5

Enable **Show Mark Labels**.

Output Visualization



Interpretation

The bar chart helps compare the average selling prices of vehicles using different fuel types.

Algorithm 4 – Pie Chart Creation

Objective

To analyze the distribution of vehicles based on fuel type.

Step 1

Drag **Fuel_Type** to the view.

Step 2

Select **Pie Chart** from Show Me.

Step 3

Drag **Car_Name** to **Angle**

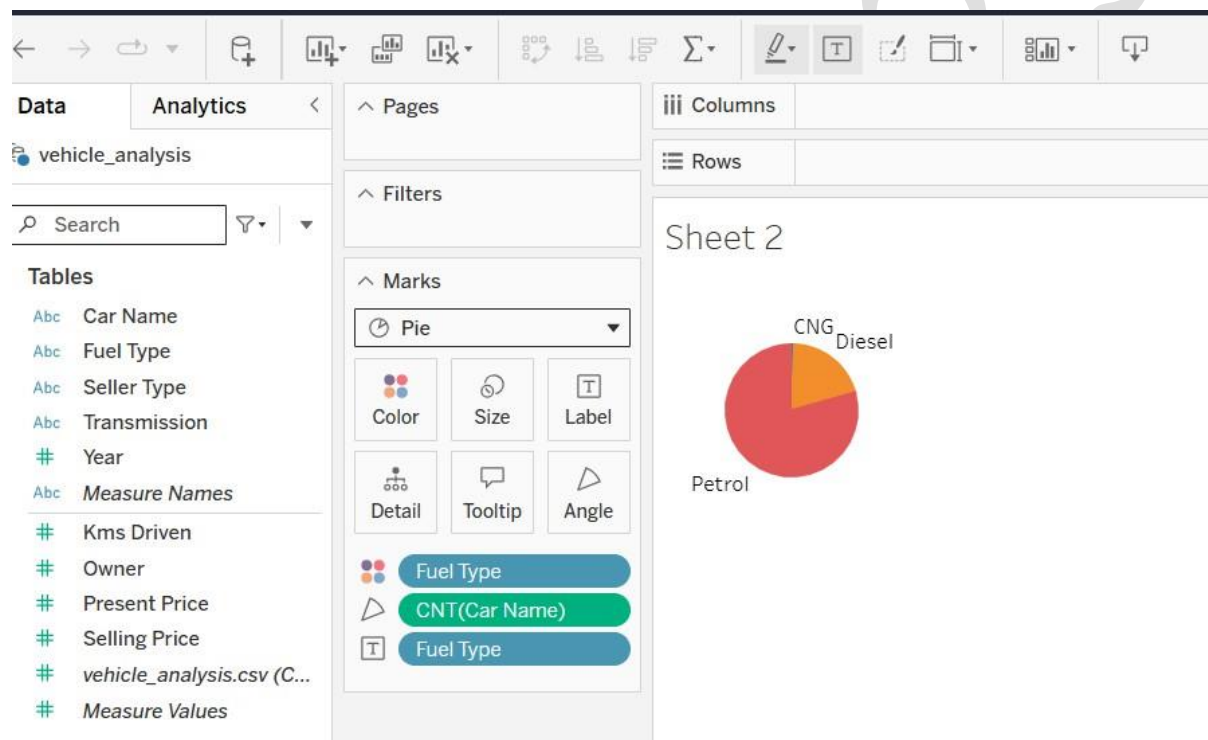
Step 4

Drag **Fuel_Type** to **Color**.

Step 5

Enable **Show Labels**.

Output Visualization



Interpretation

The pie chart shows the proportion of vehicles belonging to different fuel types such as Petrol, Diesel, and CNG.

Algorithm 5 – Line Chart Creation

Objective

To analyze the trend vehicle selling prices over the years.

Step 1

Drag **Year** → Columns.

Step 2

Drag **Selling_Price** → Rows.

Step 3

Change Selling_Price aggregation to **Average**.

Step 4

Select **Line Chart**.

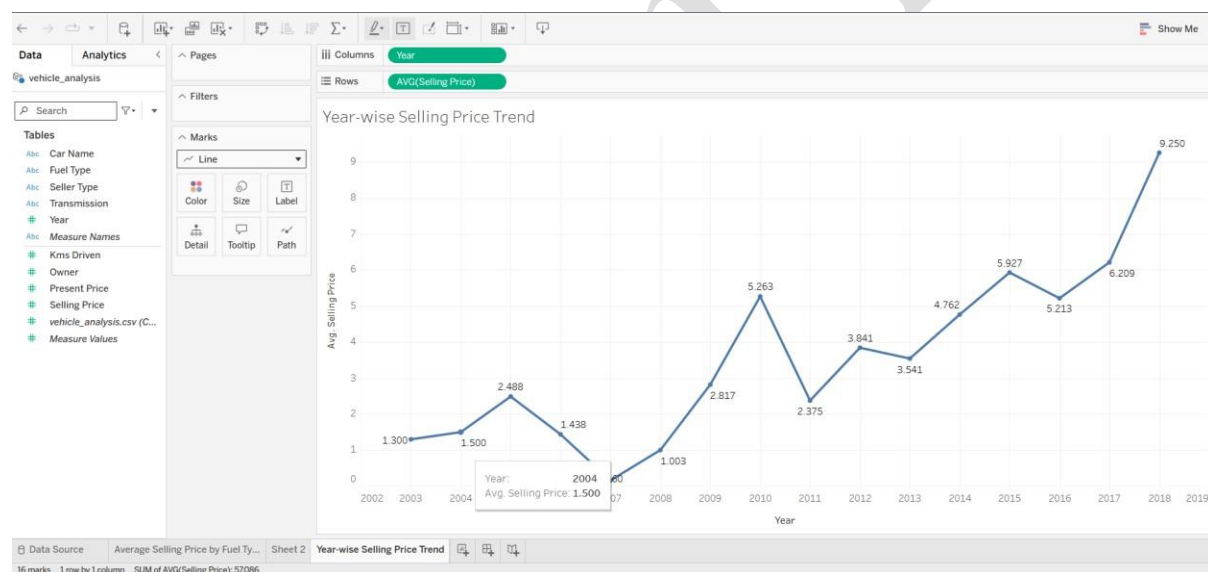
Step 5

Apply appropriate formatting.

Step 6

Analyze the trend.

Output Visualization



Interpretation

The line chart shows how the average selling price of vehicles changes across different manufacturing years.

It helps identify:

- Increasing price trends
- Decreasing price trends
- High/Low-price years
- Changes in vehicle resale value

Algorithm 6 – Creating Calculated Field

Objective

To calculate the **Vehicle Age** from the manufacturing year.

Step 1

Select:

Analysis → Create Calculated Field

Step 2

Enter the field name:

Vehicle Age

Step 3

Enter the formula:

[2026] + [Year]

Step 4

Click **OK**.

Output

Car_Name	Year	Vehicle Age
ritz	2014	12
sx4	2013	13
ciaz	2017	9
wagon r	2011	15

Interpretation

The calculated field determines the approximate age of each vehicle using its manufacturing year.

Algorithm 7 – Applying Filters

Objective

To analyze vehicles based on a specific fuel type.

Step 1

Drag **Fuel_Type** to Filters.

Step 2

Select the required fuel type.

Step 3

Apply the filter.

Step 4

Observe the updated charts.

Example

Filter: Fuel_Type = Diesel

Output

The visualization displays only vehicles using diesel fuel.

Interpretation

Filters allow users to focus on a particular category of vehicles and perform detailed analysis.

Algorithm 8 – Creating Parameter

Objective

To display vehicles based on a user-defined minimum selling price.

Step 1

Select:

Create Parameter

Step 2

Specify the parameter name:

Minimum Fatalities

Step 3

Set the data type as:

Float

Step 4

Set the required minimum value.

Step 5

Create a calculated field using the parameter.

Formula

```
[Selling_Price] >= [Minimum Selling Price]
```

Example

Minimum Selling Price = 5

Output

The visualization displays vehicles having a selling price of **5 or more**.

Interpretation

Parameters allow users to dynamically control the vehicle analysis based on a selected selling-price value.

Algorithm 9 – Dashboard Creation

Objective

To create an interactive **Vehicle Analysis Dashboard**.

Step 1

Create multiple worksheets.

Step 2

Select **Dashboard** → **New Dashboard**.

Step 3

Drag the required worksheets into the dashboard.

Step 4

Arrange the charts appropriately.

Step 5

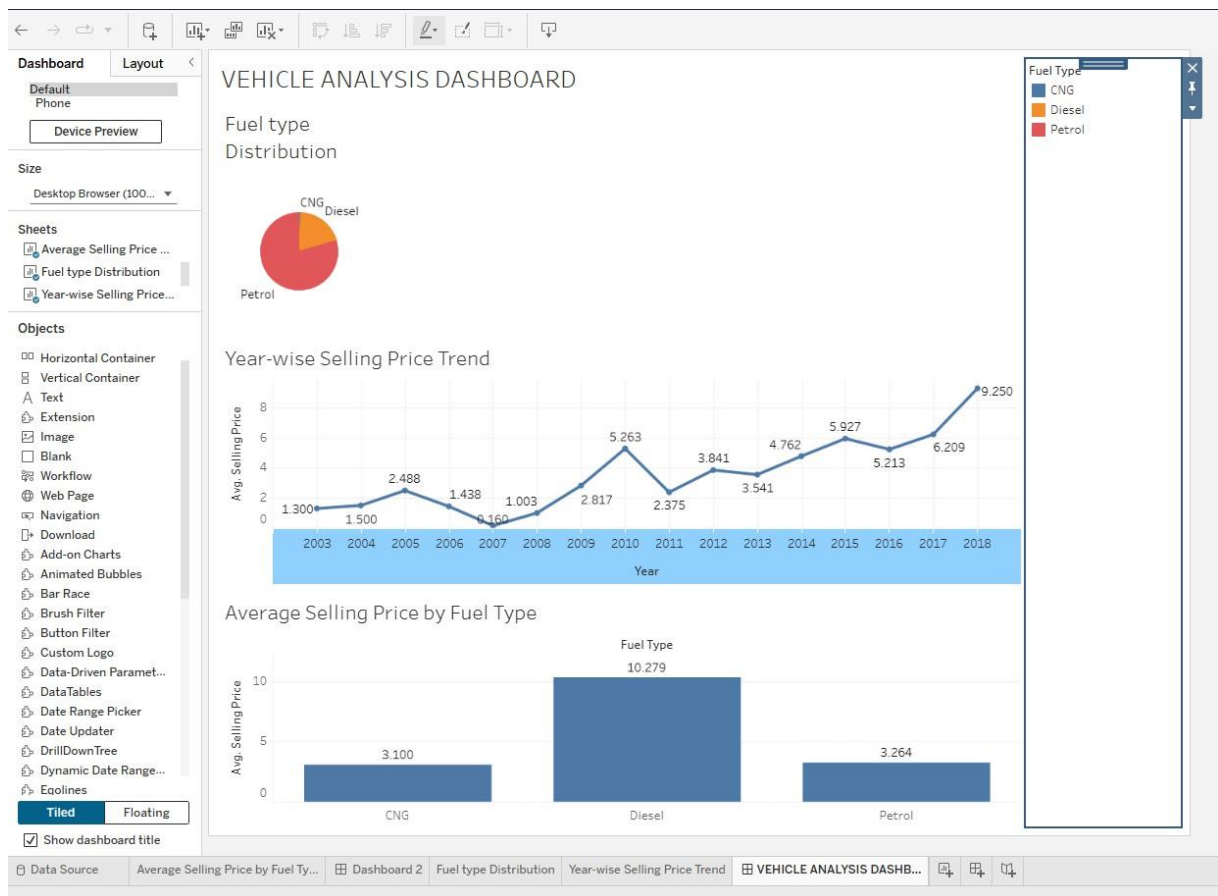
Add filters for:

- Year
- Fuel_Type
- Seller_Type
- Transmission
- Owner

Step 6

Add appropriate titles.

Dashboard Analysis



Total Vehicles

Number of vehicle records in the dataset

Average Selling Price

Average selling price of vehicles

Average Present Price

Average current/present price of vehicles

Average Kilometer Driven

Average distance driven by vehicles

Fuel Type Distribution

Distribution of vehicles according to fuel type

Transmission Distribution

Distribution of manual and automatic vehicles

Interpretation

The dashboard provides a consolidated view of vehicle characteristics, prices, usage, fuel types, and transmission types and allows users to interactively analyze the data using filters.

Algorithm 10 – Story Creation

Objective

To present the analysis of vehicle analysis through a sequence of visual insights.

Step 1

Create the required worksheets.

Step 2

Create the dashboard.

Step 3

Select **Story** → **New Story**.

Step 4

Add the dashboard and required worksheets as story points.

Step 5

Arrange the story points in a logical sequence.

Step 6

Present the findings.

Example Story

Story Point 1

Overall Vehicle Analysis

Displays total vehicles, average selling price, present price, and kilometers driven.

Story Point 2

Price Analysis

Shows the relationship and variation between vehicle selling price and present price.

Story Point 3

Fuel Type Analysis

Shows the distribution of vehicles based on Petrol, Diesel, and CNG.

Story Point 4

Vehicle Usage Analysis

Analyzes vehicles based on kilometers driven.

Story Point 5

Transmission Analysis

Compares vehicles with manual and automatic transmission.

Story Point 6

Final Conclusions

Summarizes the major patterns and observations obtained from the Vehicle Analysis Dataset.

Applications

Automobile Industry

Vehicle price and market analysis.

Used Car Market

Analysis of resale prices and vehicle demand.

Sales Analysis

Comparison of selling prices across different vehicle categories.

Customer Analysis

Understanding fuel and transmission preferences.

Business Intelligence

Interactive vehicle sales dashboards and reports.

Data Analytics

Exploratory analysis of vehicle characteristics and pricing trends.

Result

Ex.no:8	MINI PROJECT – HEALTHCARE ANALYTICS DASHBOARD

Domain and Problem Statement:

Domain: Healthcare Analytics

Problem Statement: Healthcare organizations need better insights into patient records, medical conditions, treatment patterns, and hospital performance. This project develops an interactive dashboard to analyze healthcare data and support data-driven decisions.

Dataset Description:

Dataset Name: Healthcare Patient Dataset

Granularity: Each row represents a patient record.

Key Entities & Attributes: Patient ID, Age, Gender, Diagnosis, Treatment, Hospital, Admission Date, Billing Amount and Medical Condition.

Tools Used

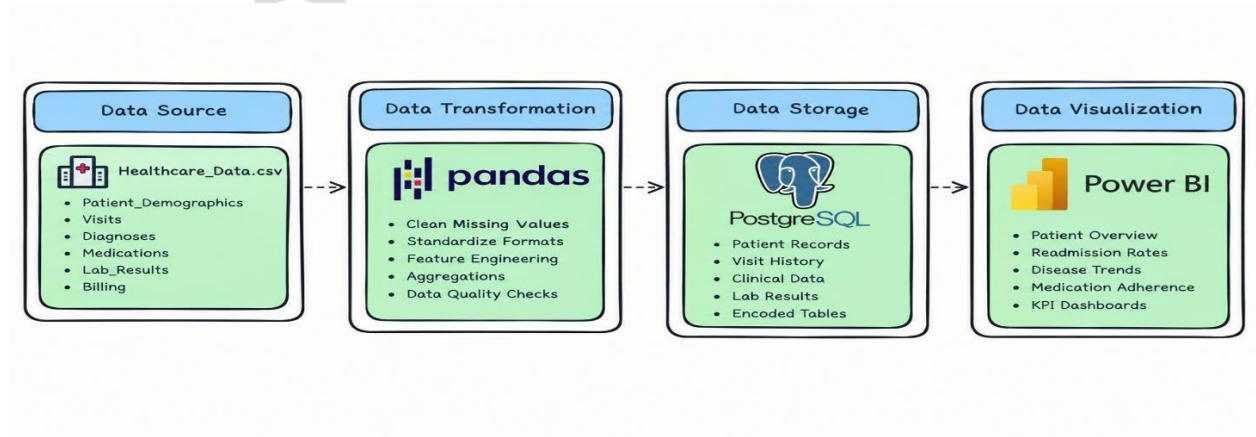
Python, Pandas, PostgreSQL, Power BI and Tableau.

Data Source:

Primary Source: Kaggle Open Datasets (Healthcare analytics Dataset).

Storage Infrastructure: Cleaned transactional records hosted within a local SQL database instance (healthcare_analytics).

Architecture:



Program:

1. Python – Data Cleaning & Transformation

```
import pandas as pd

from sqlalchemy import create_engine, text

# Load healthcare dataset
df = pd.read_csv("healthcare_data.csv")

healthcare = df.copy()

# Remove unnecessary columns
# Modify these columns according to your dataset
healthcare.drop(
    columns=["doctor_name", "hospital_address"],
    errors="ignore",
    inplace=True
)

# Handle missing values
healthcare["age"] = healthcare["age"].fillna(
    healthcare["age"].median()
)

healthcare["gender"] = healthcare["gender"].fillna("Unknown")
healthcare["diagnosis"] = healthcare["diagnosis"].fillna("Unknown")
healthcare["medication"] = healthcare["medication"].fillna("Unknown")

# Remove duplicate patient/visit records
healthcare.drop_duplicates(inplace=True)

healthcare["admission_date"] = pd.to_datetime(
    healthcare["admission_date"],
    errors="coerce"
)

healthcare["discharge_date"] = pd.to_datetime(
    healthcare["discharge_date"],
    errors="coerce"
)

healthcare["length_of_stay"] = (
    healthcare["discharge_date"] -
    healthcare["admission_date"]
```

```
).dt.days
# Create age groups
healthcare["age_group"] = pd.cut(
    healthcare["age"],
    bins=[0, 18, 35, 50, 65, 100],
    labels=[
        "0-18",
        "19-35",
        "36-50",
        "51-65",
        "65+"
    ]
)
if "bmi" in healthcare.columns:
    healthcare["bmi_category"] = pd.cut(
        healthcare["bmi"],
        bins=[0, 18.5, 25, 30, 100],
        labels=[
            "Underweight",
            "Normal",
            "Overweight",
            "Obese"
        ]
    )
if "billing_amount" in healthcare.columns:
    healthcare["billing_category"] = pd.cut(
        healthcare["billing_amount"],
        bins=[0, 10000, 50000, 100000, float("inf")],
        labels=[
            "Low",
            "Medium",
            "High",
            "Very High"
        ]
    )
```

```

# Filter invalid records
# Remove records with invalid age
healthcare = healthcare[
    (healthcare["age"] >= 0) &
    (healthcare["age"] <= 100)
]
healthcare = healthcare[
    healthcare["length_of_stay"].fillna(0) >= 0
]
if "billing_amount" in healthcare.columns:
    healthcare = healthcare[
        healthcare["billing_amount"].fillna(0) >= 0
    ]
healthcare.columns = (
    healthcare.columns
    .str.lower()
    .str.strip()
    .str.replace(" ", "_")
)
print("Original Shape:", df.shape)
print("Cleaned Shape:", healthcare.shape)
print("\nMissing Values:")
print(healthcare.isnull().sum())
print("\nCleaned Dataset:")
print(healthcare.head())

```

2. MySQL – Database and Data Loading

net start MySQL80

mysql -u root -p

Create Healthcare Analytics Database

```
CREATE DATABASE IF NOT EXISTS healthcare_analytics;
```

```
USE healthcare_analytics;
```

```
name = "root"
```

```
password = "YOUR_MYSQL_PASSWORD"
```

```
host = "localhost"
```

```
    f"mysql+pymysql://{username}:{password}@{host}:{port}/hotel_analytics"  
)
```

```
# Load cleaned data into MySQL
```

```
hotel.to_sql(  
    "hotel_bookings",  
    con=db_engine,  
    if_exists="replace",  
    index=False,  
    chunksize=5000  
)
```

```
print("Data loaded successfully into MySQL.")
```

Verify:

```
pd.read_sql(  
    "SELECT COUNT(*) AS total_rows FROM hotel_bookings",  
    db_engine  
)
```

Procedure:

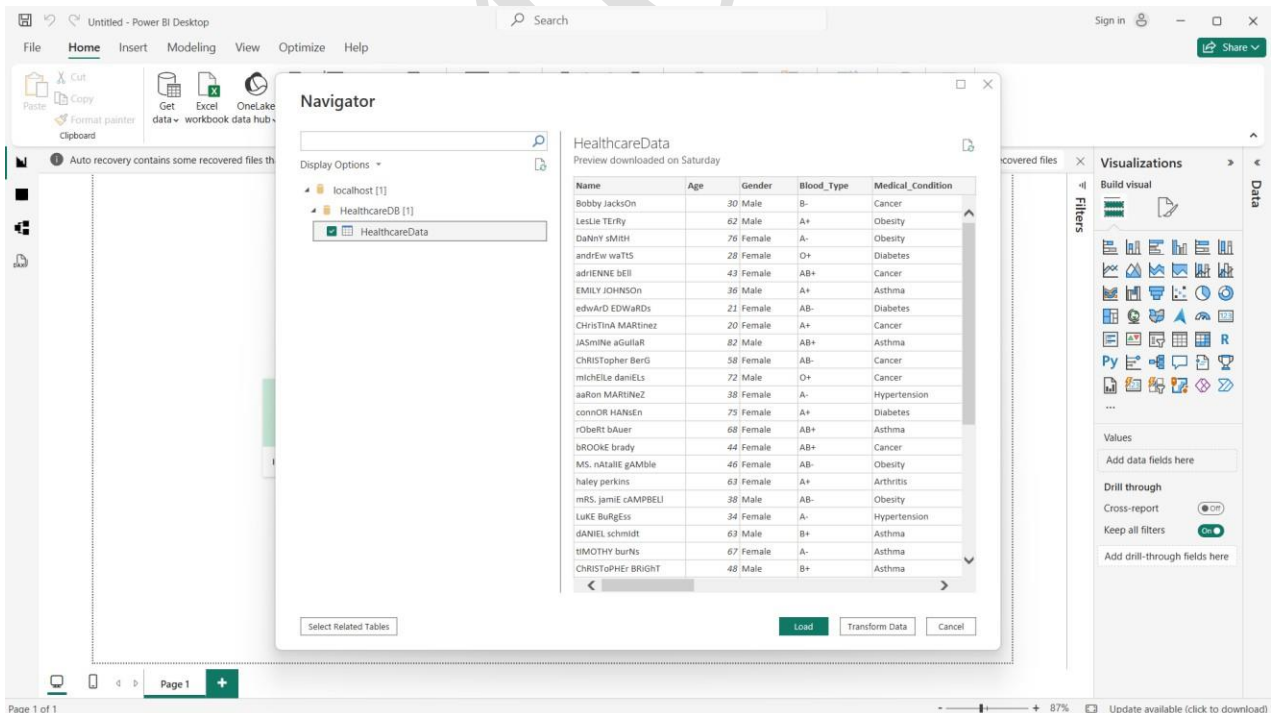
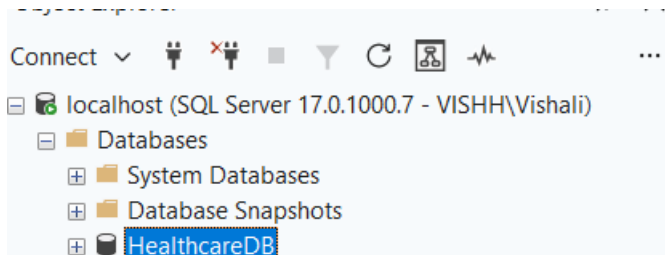
Step 1: Data Preparation & Preprocessing (Python / Pandas)

- Load the raw healthcare dataset into a Jupyter Notebook or Python script using `pandas.read_csv()`.
- Inspect the dataset for null values and handle missing data in important healthcare fields such as:
 - Age
 - Gender
 - Diagnosis
 - Medication
 - Clinical measurements
- Remove duplicate patient or healthcare visit records.

- Convert admission and discharge dates into proper datetime format.
- Feature engineer **length of hospital stay**
- Filter invalid records such as: Negative age, Age greater than 100, Negative hospital stay
- Standardize column names by converting them to lowercase and replacing spaces with underscores.
- Perform final data-quality checks and export the cleaned DataFrame to MySQL using SQLAlchemy and to_sql.

Step 2: SQL Database Connection & Data Ingestion

1. Open Power BI Desktop → Click **Get Data** → Select **MySQL Database**.
2. Enter the **Server** (localhost:3306) and **Database Name** (healthcare_analytics).
3. Enter authentication credentials, select the cleaned flat table (healthcare_analytics), and click **Transform Data** to open Power Query



Step 3: (Power BI) – Healthcare_Analytics

After loading the cleaned data into MySQL, connect **Power BI** to the healthcare_analytics database.

Patient Overview

- Total Patients
- Male vs Female Patients
- Average Patient Age
- Patient Distribution by Age Group

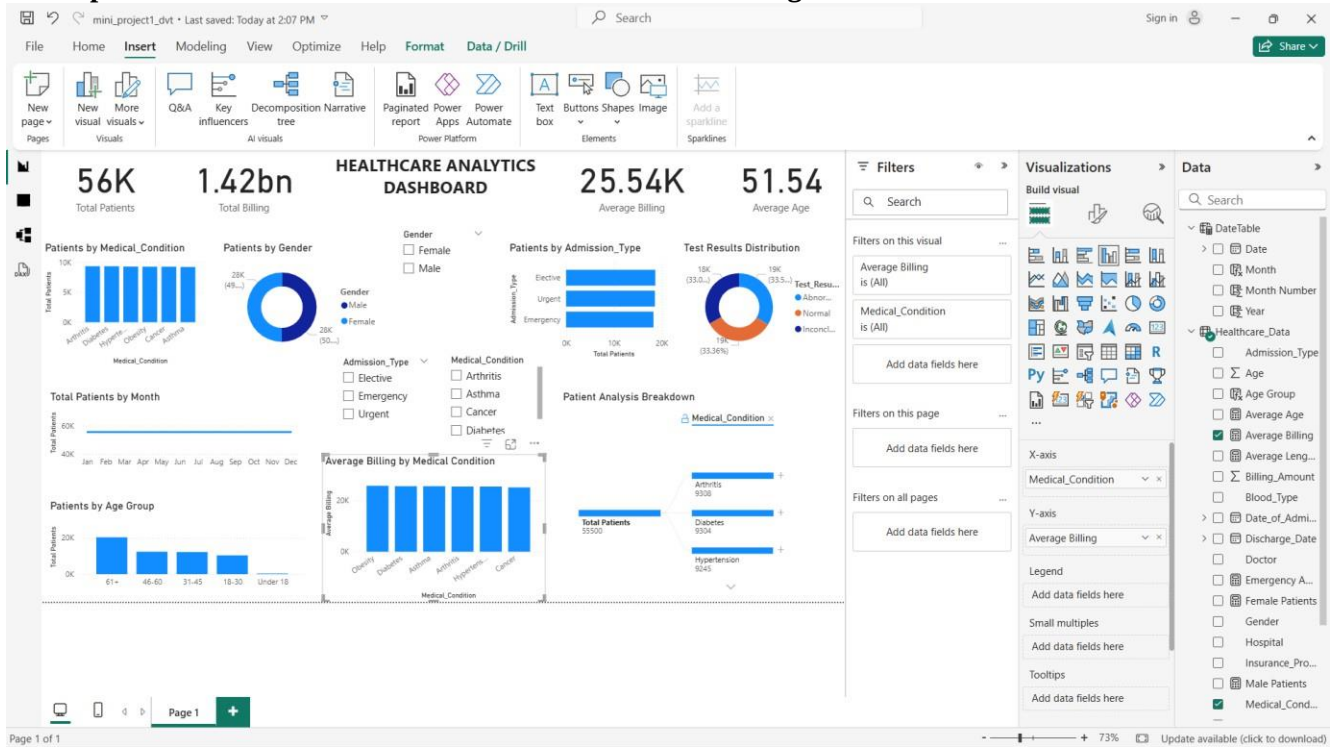
Hospital Analytics

- Total Admissions
- Average Length of Stay
- Admissions by Month
- Hospital/Department Performance.

	Name	Age	Gender	Blood_Type	Medical_Condition	Date_of_Admission	Doctor
1	Bobby JacksOn	30	Male	B-	Cancer	31-01-2024	Matthew Sml
2	Leslie TERry	62	Male	A+	Obesity	20-08-2019	Samantha Da
3	DaNy SMITH	76	Female	A-	Obesity	22-09-2022	Tiffany Mitch
4	andrew waTis	28	Female	O+	Diabetes	18-11-2020	Kevin Wells
5	adriENNE beil	43	Female	AB+	Cancer	19-09-2022	Kathleen Han
6	EMILY JOHNSOn	36	Male	A+	Asthma	20-12-2023	Taylor Neuto
7	edward EDWards	21	Female	AB-	Diabetes	03-11-2020	Kelly Olson
8	ChrisTina MARTinez	20	Female	A+	Cancer	28-12-2021	Suzanne Thor
9	JASmiNe aGullaR	82	Male	AB+	Asthma	01-07-2020	Daniel Fergus
10	CHRISTopher BERG	58	Female	AB-	Cancer	23-05-2021	Heather Day
11	michELLE daniELS	72	Male	O+	Cancer	19-04-2020	John Duncan
12	aallon MARTINEZ	38	Female	A-	Hypertension	13-08-2023	Douglas Mayr
13	connOR HANsEn	75	Female	A+	Diabetes	12-12-2019	Kenneth Flet

Field Name
Admission_Type
Age
Age Group
Average Age
Average Billing
Average Leng...
Billing_Amount
Blood_Type
Date_of_Admi...
Discharge_Date
Doctor
Emergency A...
Female Patients
Gender
Hospital
Insurance_Pro...
Male Patients
Medical_Cond...
Medication
Name
Room_Number
Test_Results

Step 4: Power BI Final Executive Dashboard & Drill-Through



1. Executive Canvas:

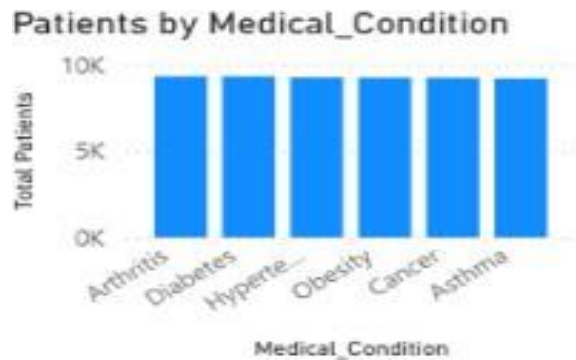
- Place **Card Visuals** across the top: Total Patients, Total Billings, Average Age, Average Billing.



- Add a **Line Chart** mapped to Dim_Date[date] with hierarchy drill-down enabled (Year → Quarter → Month).



- Add a **Clustered Bar Chart** for Medical_Condition

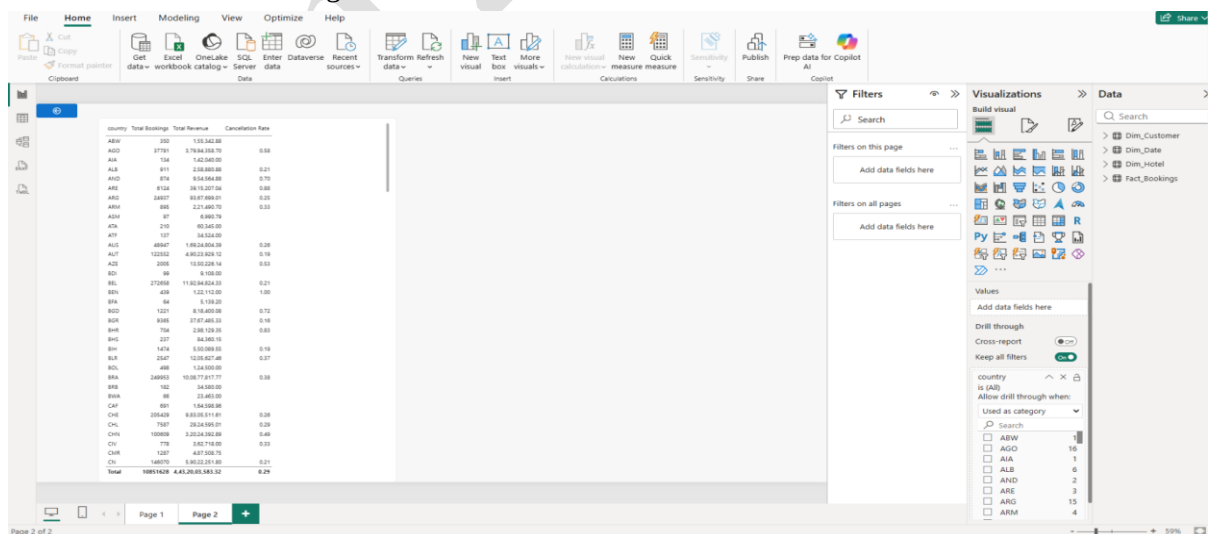


- Add **Slicers** for Gender, Admission type, Medical Condition.



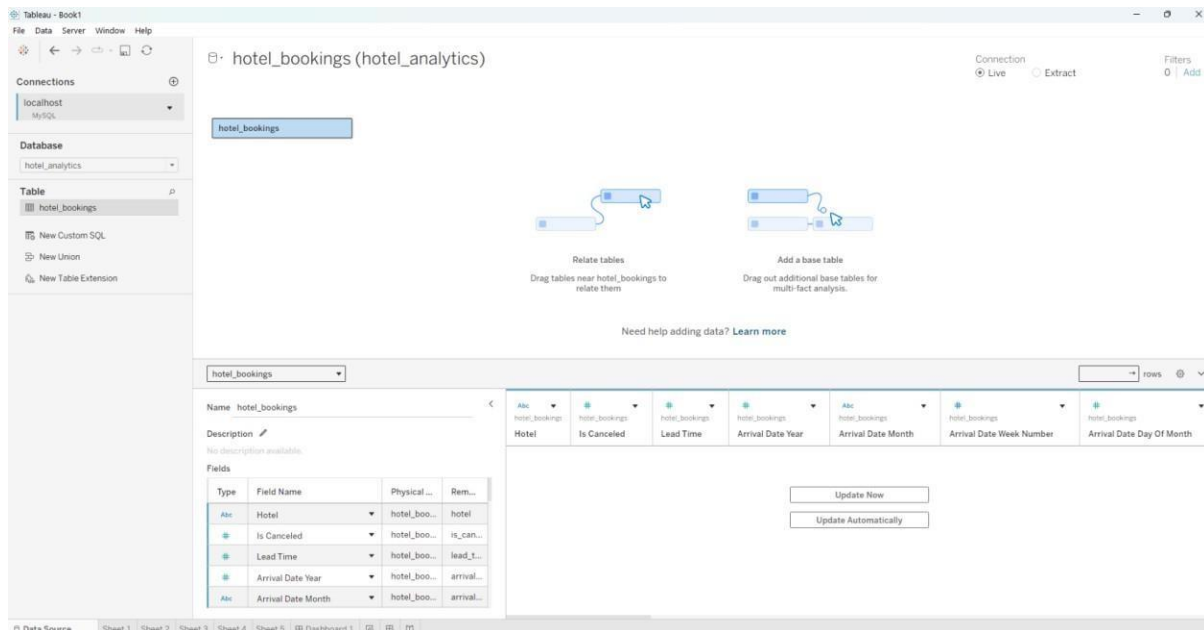
2. Country Details Drill-Through Page:

- Create a second page named Country Details.
- Drag Dim_Customer[country] into the **Drill-through** field bucket.
- Add a detailed Matrix/Table visual showing country, Total Bilings, Total Revenue, and Average Rate



Step 6: Tableau Data Connection & Preparation

1. Open Tableau Desktop / Tableau Public.
2. Under **To a Server**, select **MySQL**
3. Enter Server (localhost), Database (healthcare_analytics), and user credentials to pull the cleaned dataset onto the canvas.



- **Select Metric Parameter:** String list containing "Total Revenue", "Total Billings"

Edit Parameter [Select Metric]

Name: Select Metric

Properties

Data type: String

Display format: Total Revenue

Current value: Total Revenue

Value when workbook opens: Current value

Allowable values

☐ All ☒ List ☐ Range

Value	Display As
Total Revenue	Total Revenue
Total Bookings	Total Bookings
Cancellation Rate	Cancellation Rate
Click to add	

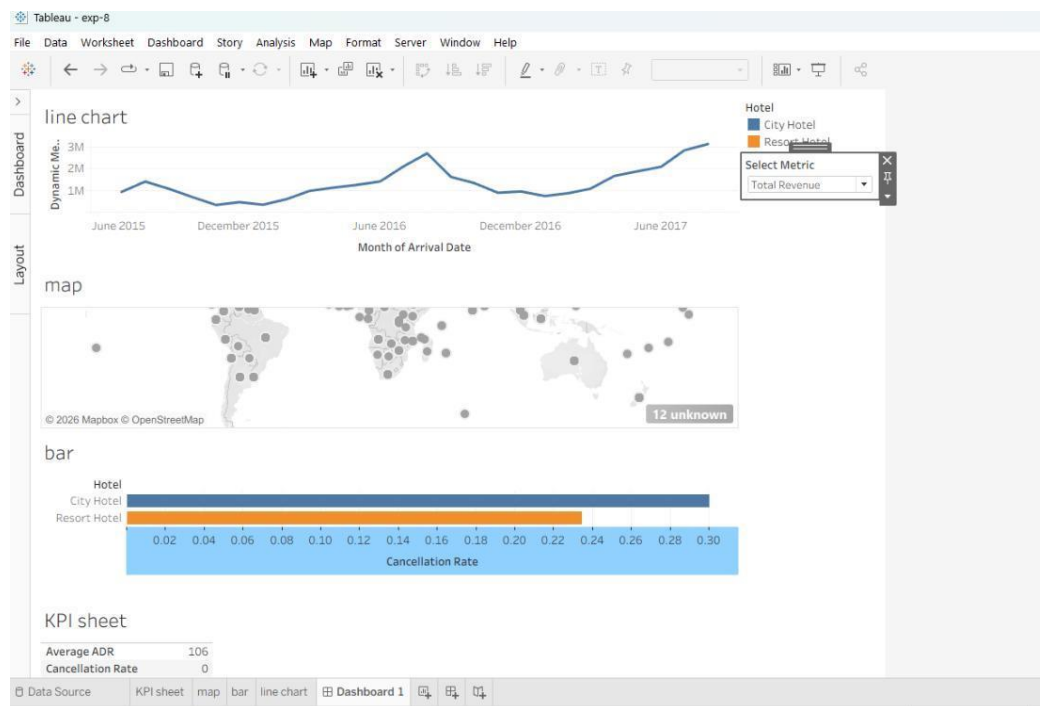
☒ Fixed ☐ When workbook opens

Add values from

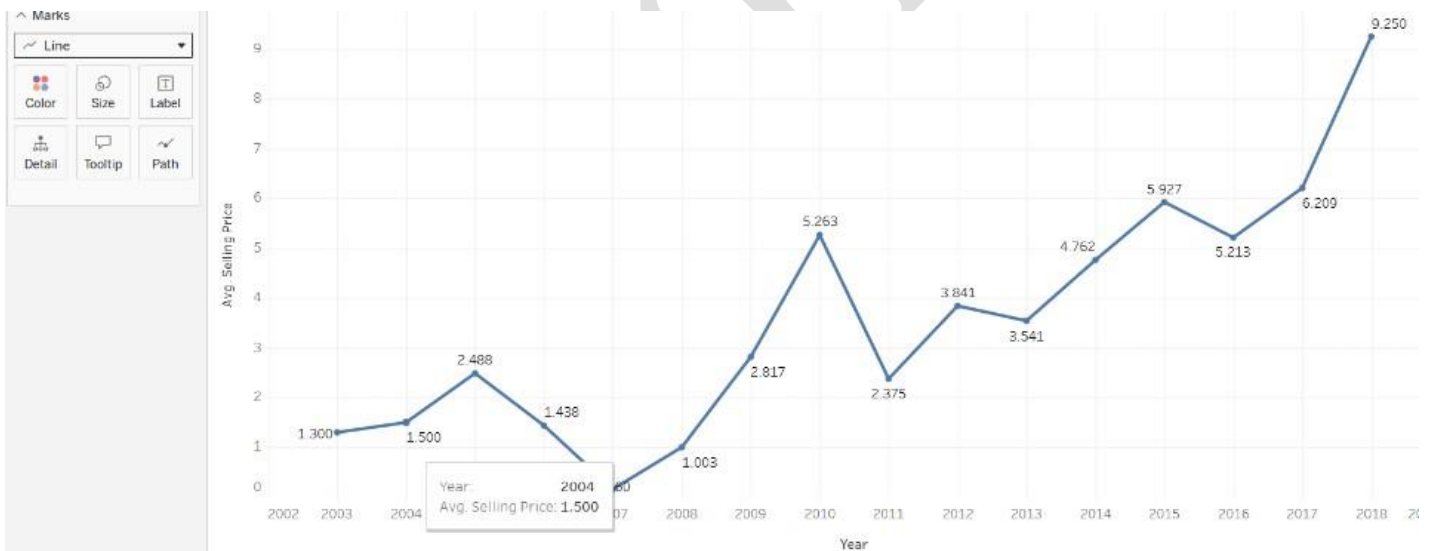
Remove Selected

Cancel OK

Step 7: Tableau Companion Dashboard & Interactive Story



1. **Worksheets:** Build KPI Cards, Revenue Trend Line Chart (using Dynamic Measure), Hotel Cancellation Bar Chart, and Country Map.

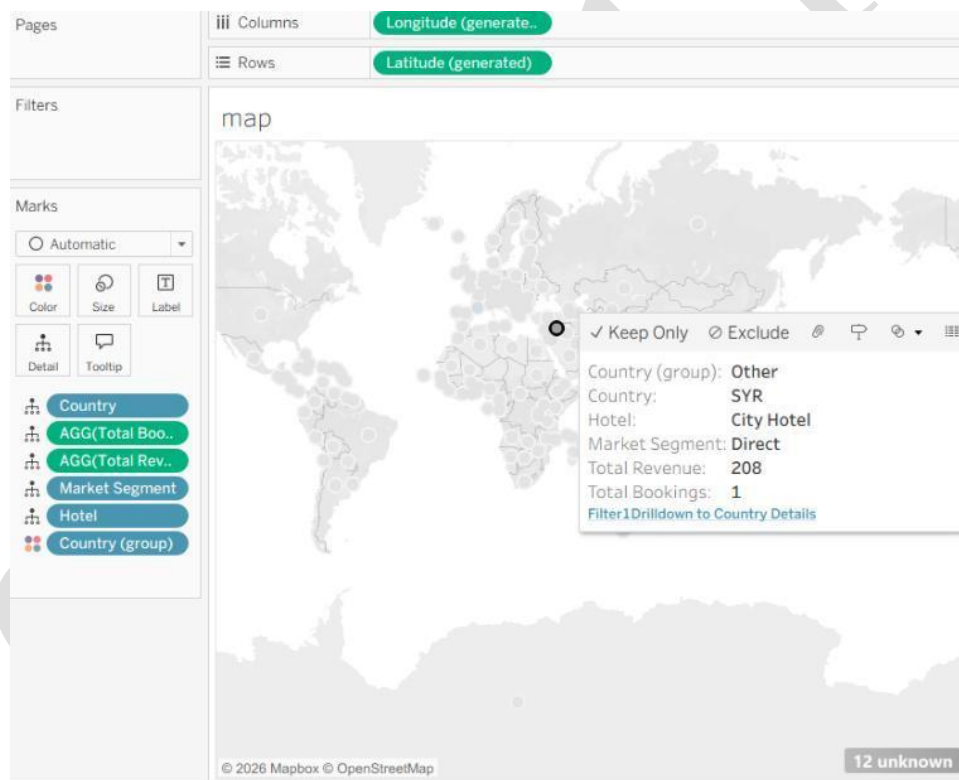


2. Global Filters & Actions:

- Drag Hotel, Deposit Type, and Arrival Date to Filters → set to **Apply to Worksheets** → **All Using This Data Source**.
- Drag sheets onto a new Dashboard tab and click the **Use as Filter** funnel icon on the Country Map visual.



3. Drill-down map:



Observed Insights:

- **Patient Volume & Healthcare Utilization:** The dataset captures overall patient records and hospital visits, providing insights into healthcare demand and utilization.
- **Patient Demographics:** Patient distribution varies across age groups and gender, highlighting key demographic segments requiring healthcare services.
- **Disease & Diagnosis Trends:** Common diseases and diagnoses account for a major share of patient visits, helping identify prevalent health conditions.
- **Treatment & Hospital Stay:** Length of stay and treatment patterns vary by diagnosis, indicating differences in healthcare resource utilization.
- **Healthcare Cost Analysis:** Medical expenses and billing amounts increase with complex treatments and longer hospital stays, highlighting major cost drivers.

RESULT:

Ex.no:9	MINI PROJECT – FINANCE & BANKING ANALYTICS

Domain and Problem Statement:

Domain: Banking Analytics & Financial Fraud Detection

Problem Statement: Financial institutions face capital loss and operational risk due to undetected transaction fraud, inefficient channel utilization, and poor visibility into state-level liquidity distribution. This project addresses these challenges by transforming raw transaction records into structured business intelligence across Power BI and Tableau to evaluate financial performance, geographic risk distribution, and fraud exposure across banking channels.

Dataset Description:

Dataset Name: Indian Banking Transactions Dataset

Granularity: Each row represents a single banking transaction.

Timeframe Covered: 2019 to 2024.

Key Entities & Attributes:

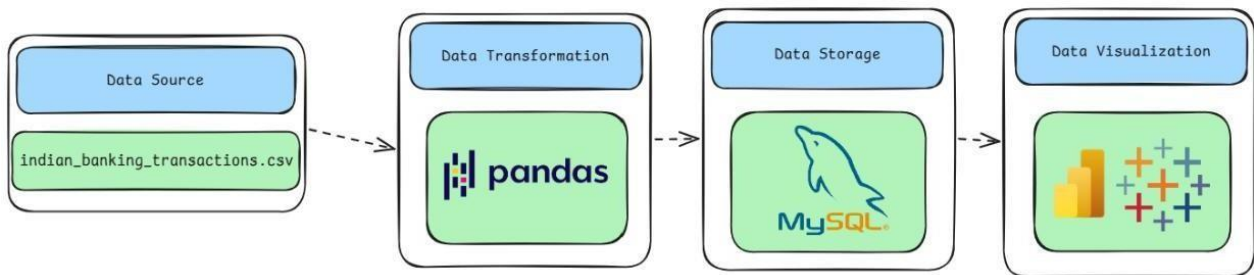
- Demographics & Account:** customer_id, state, credit_score, has_loan, loan_type, kyc_status, account_type, channel.
- Transaction Details:** transaction_id, transaction_date, transaction_time, transaction_hour, transaction_type, transaction_direction, merchant_category, transaction_status.
- Financials & Risk:** transaction_amount, account_balance, emi_amount, is_fraud.

Data Source:

Primary Source: Kaggle Open Datasets (Indian Banking Transactions).

Storage Infrastructure: Cleaned transactional records hosted within a local MySQL database instance (banking_analytics).

Architecture:



Program:

1. Python – Data Cleaning & Transformation

```
import pandas as pd

from sqlalchemy import create_engine, text

# Load dataset

df = pd.read_csv("indian_banking_transactions.csv")

bank = df.copy()


# Handle missing values

bank["customer_id"] = bank["customer_id"].fillna("Unknown")

bank["transaction_type"] = bank["transaction_type"].fillna("Unknown")

bank["account_type"] = bank["account_type"].fillna("Unknown")

bank["merchant_category"] = bank["merchant_category"].fillna("Unknown")

bank["transaction_direction"] = bank["transaction_direction"].fillna("Unknown")

bank["transaction_amount"] = bank["transaction_amount"].fillna(0)

bank["account_balance"] = bank["account_balance"].fillna(0)


# Remove duplicates

bank.drop_duplicates(inplace=True)


# Convert date column

bank["transaction_date"] = pd.to_datetime(
    bank["transaction_date"], errors="coerce"
)


# Standardize column names

bank.columns = (
    bank.columns.str.lower().str.strip().str.replace(" ", "_")
)

print("Original Shape:", df.shape)

print("Cleaned Shape:", bank.shape)

bank.head()
```

2. MySQL – Database and Data Loading

MySQL Command Prompt:

```
net start MySQL80
```

```
mysql -u root -p
```

MySQL:

```
CREATE DATABASE IF NOT EXISTS hotel_analytics;
```

Python/Jupyter:

```
# Database Server Credentials
```

```
username = "root"
```

```
password = "YOUR_MYSQL_PASSWORD"
```

```
host = "localhost"
```

```
port = 3306
```

```
# Create Database
```

```
server_engine = create_engine(
```

```
    f"mysql+pymysql://{username}:{password}@{host}:{port}"
```

```
)
```

```
with server_engine.connect() as connection:
```

```
    connection.execute(
```

```
        text("CREATE DATABASE IF NOT EXISTS banking_analytics")
```

```
)
```

```
# Connect to target DB and export table
```

```
db_engine = create_engine(
```

```
    f"mysql+pymysql://{username}:{password}@{host}:{port}/banking_analytics"
```

```
)
```

```
bank.to_sql(
```

```
    "banking_transactions",
```

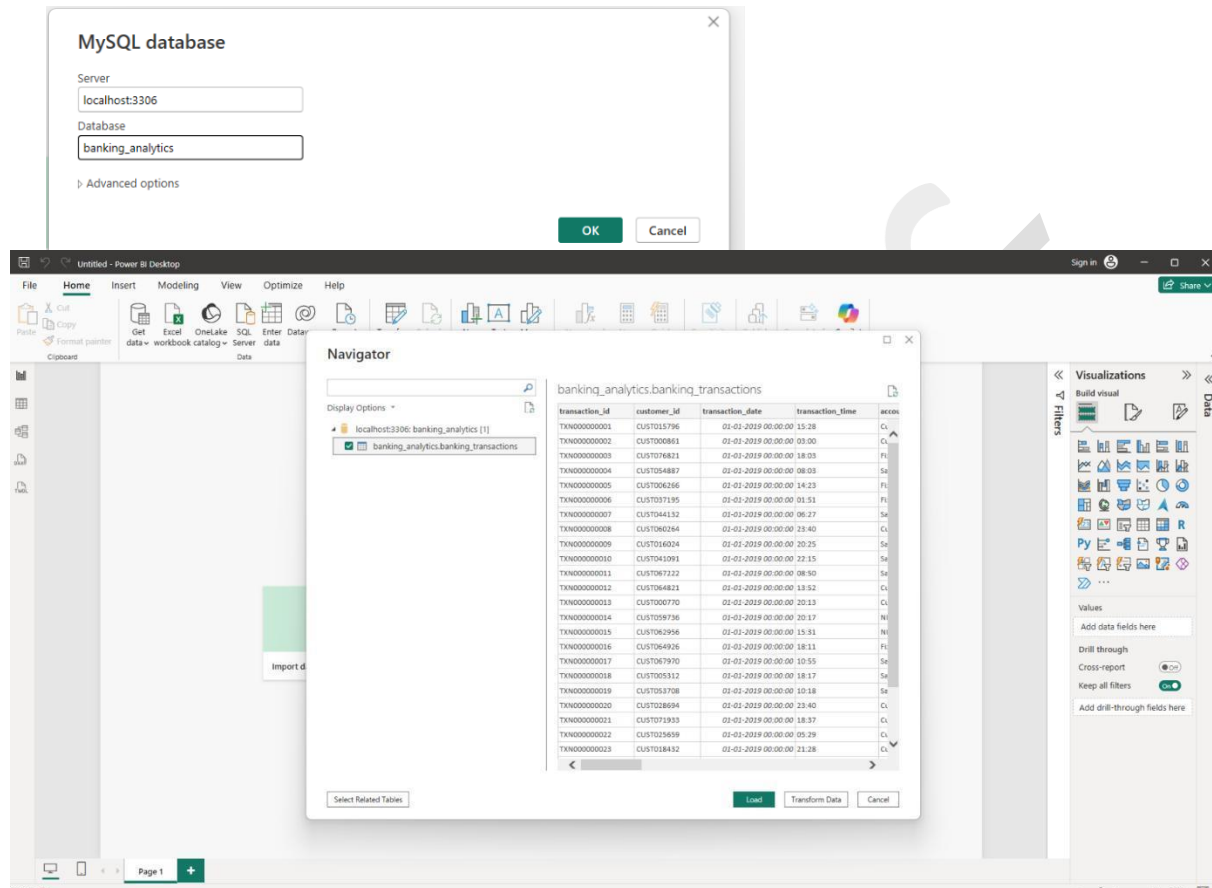
```
    con=db_engine,
```

```
    if_exists="replace",
```

```
    index=False,
```


Step 2: SQL Database Connection & Data Ingestion

1. Open Power BI Desktop → Click **Get Data** → Select **MySQL Database**.
2. Enter Server (localhost:3306) and Database Name (banking_analytics).
3. Authenticate with root credentials, select banking_transactions, and click **Transform Data** to launch Power Query.



Step 3: Star Schema Data Modeling (Power BI)

1. Table Splitting in Power Query:

Duplicate raw query banking_transactions into 3 separate entities: Fact_Transactions, Dim_Customer, and Dim_Account.

Dim_Customer: Retain customer_id, state, credit_score, has_loan, loan_type, kyc_status. Remove duplicates on customer_id (79,916 unique rows).

Dim_Account: Retain account_type and channel. Remove duplicates and add an Index Column named **AccountKey**.

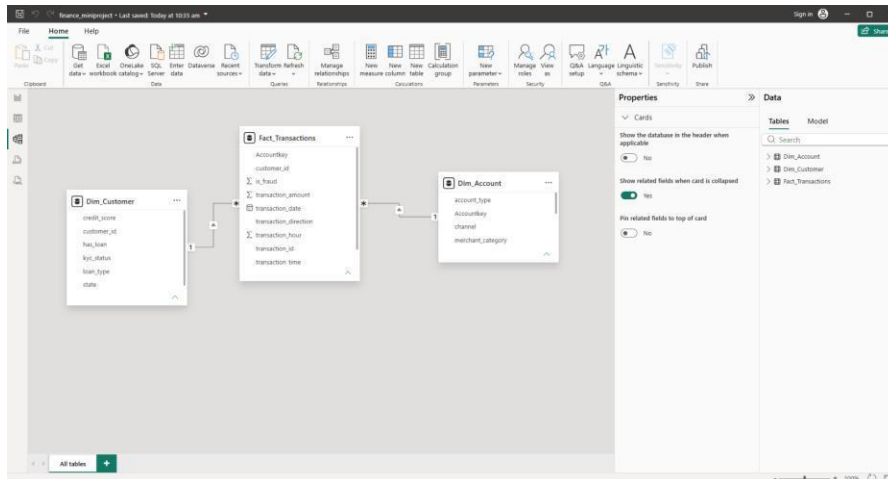
Fact_Transactions: Merge with Dim_Account on account_type and channel to pull in AccountKey. Then remove redundant text columns (state, credit_score, has_loan, loan_type, kyc_status, account_type, channel).

2. Establishing Relationships in Model View:

Connect Dim_Customer[customer_id] → Fact_Transactions[customer_id] (1:*).

Connect Dim_Account[AccountKey] → Fact_Transactions[AccountKey] (1:*).

Hide foreign key columns inside Fact_Transactions to enforce clean report pane structure.



Step 4: Core Calculations & Measures (DAX)

Power BI Measures (DAX)

Create a dedicated _Measures table and add the following DAX calculations:

- **Total Transactions:**

```
1 Total Transactions = COUNTROWS(Fact_Transactions)
```

- **Total Transaction Amount:**

```
1 Total Transaction Amount = SUM(Fact_Transactions[transaction_amount])
```

- **Average Transaction:**

```
1 Average Transaction = AVERAGE(Fact_Transactions[transaction_amount])
```

- **Total Customers:**

```
1 Total Customers = DISTINCTCOUNT(Fact_Transactions[customer_id])
```

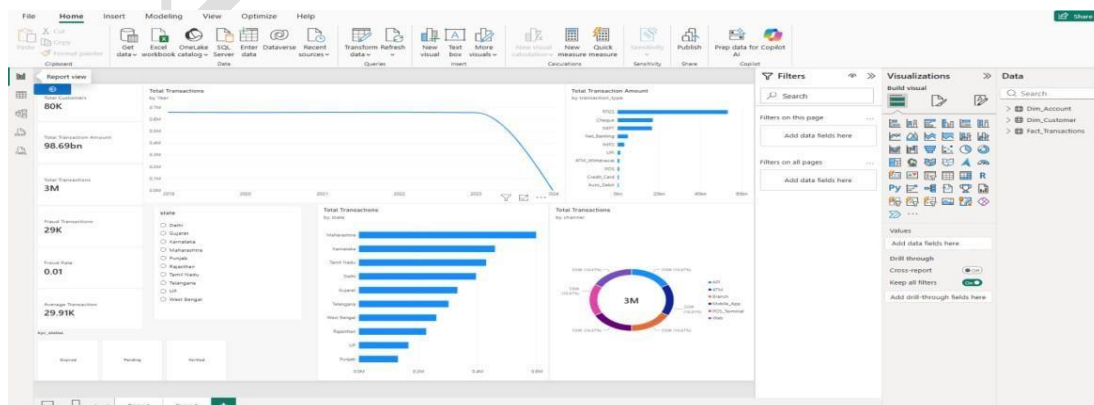
- **Fraud Transactions:**

```
1 Fraud Transactions = CALCULATE(COUNTROWS(Fact_Transactions), Fact_Transactions[is_fraud] = 1)
```

- **Fraud Rate (%):**

```
Fraud Rate = DIVIDE([Fraud Transactions], [Total Transactions], 0)
```

Step 5: Power BI Final Executive Dashboard & Drill-Through

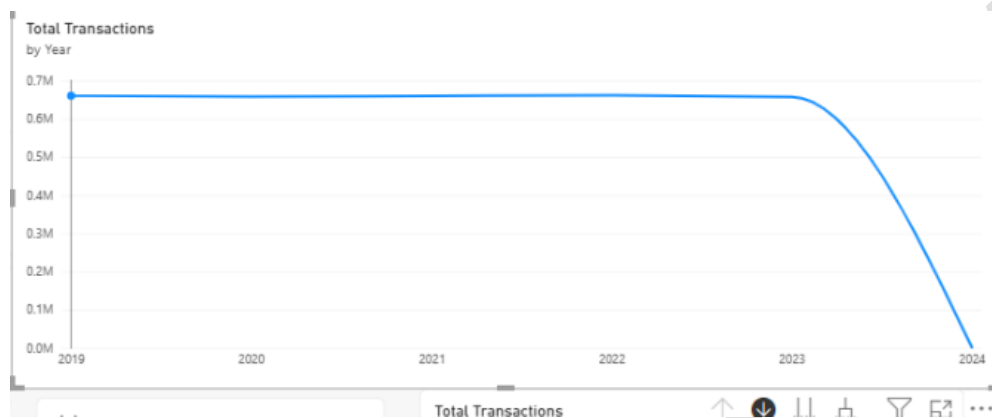


1. Executive Canvas:

- Place **Card Visuals** across the top: Total Customers, Total Transaction Amount, Total Transactions, Fraud Transactions, Fraud Rate, Average Transaction

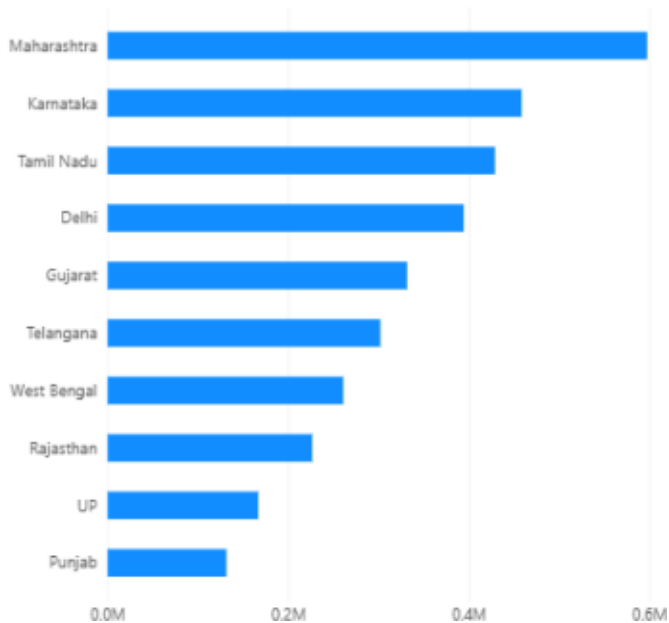


- Add a **Line Chart** mapped to Total Transactions by Year.

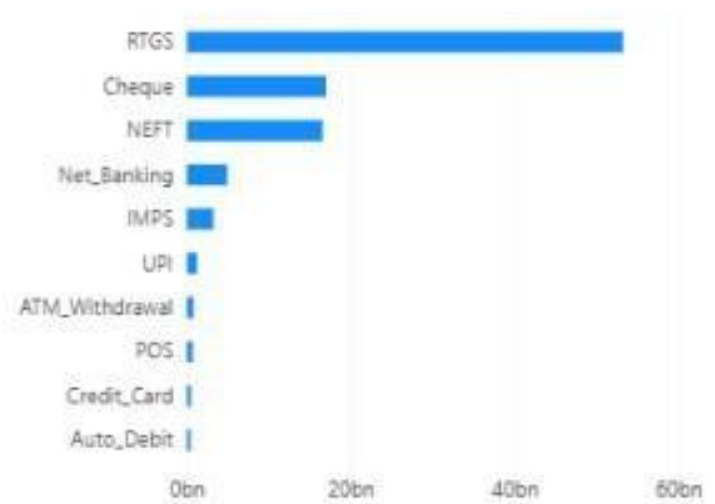


- Add a **Clustered Bar Chart** for Total Transactions by state

Total Transactions
by state

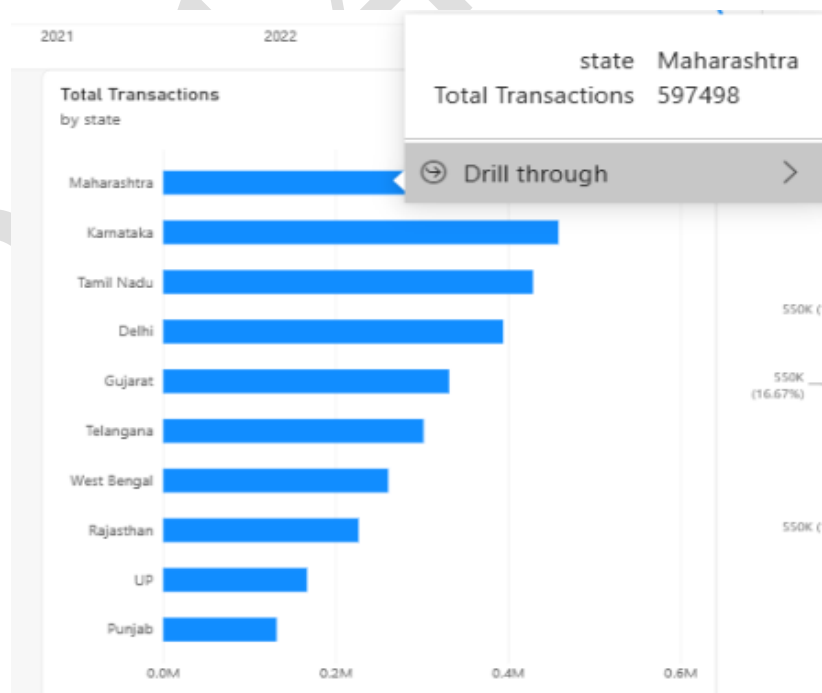
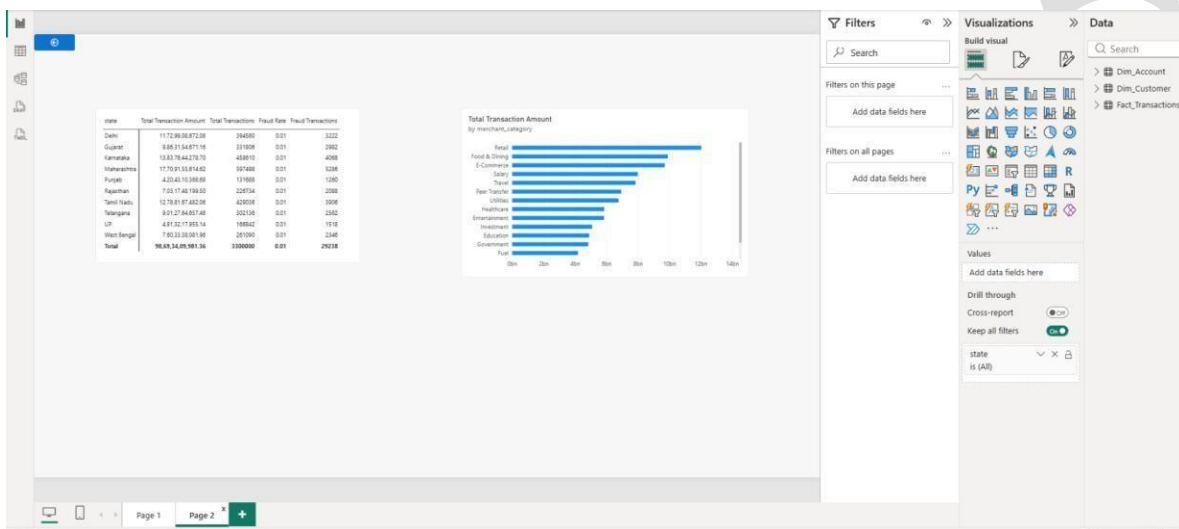


Total Transaction Amount
by transaction_type



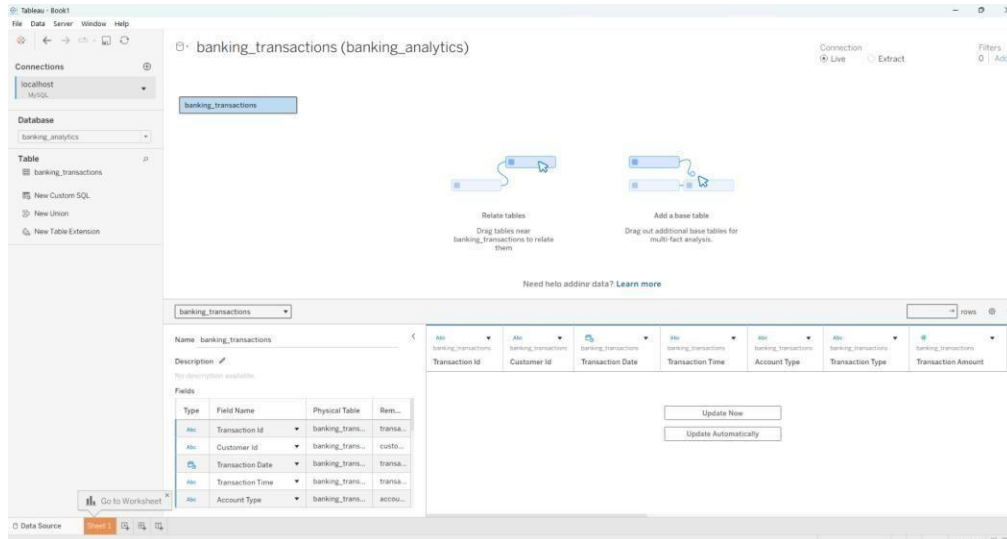
2. State Details Drill-Through Page:

- Create a second page named Country Details.
- Drag Dim_Customer[country] into the **Drill-through** field bucket.
- Add a detailed Matrix/Table visual showing country, Total Bookings, Total Revenue, and Cancellation Rate
- Create a second page named Page 2.
- Drag Dim_Customer[state] into the Drill-through field bucket.
- Add a detailed Matrix/Table visual showing state, Total Transaction Amount, Total Transactions, Fraud Rate, and Fraud Transactions.
- Add a Bar Chart for Total Transaction Amount by merchant_category.



Step 6: Tableau Data Connection & Preparation

1. Open Tableau Desktop / Tableau Public.
2. Under **To a Server**, select **MySQL**.
3. Enter Server (localhost), Database (banking_analytics), and user credentials to pull the cleaned dataset onto the canvas.



1. Go to any sheet and build calculated fields:

- **Total Transaction amount:**

Total Transaction amount  banking_transactions (banking_analytics)

```
SUM([Transaction Amount])
```

- **Total trnsactions:**

Total transactions  banking_transactions (banking_analytics)

```
COUNT([Transaction Id])
```

- **Fraud rate:**

Fraud rate  banking_transactions (banking_analytics)

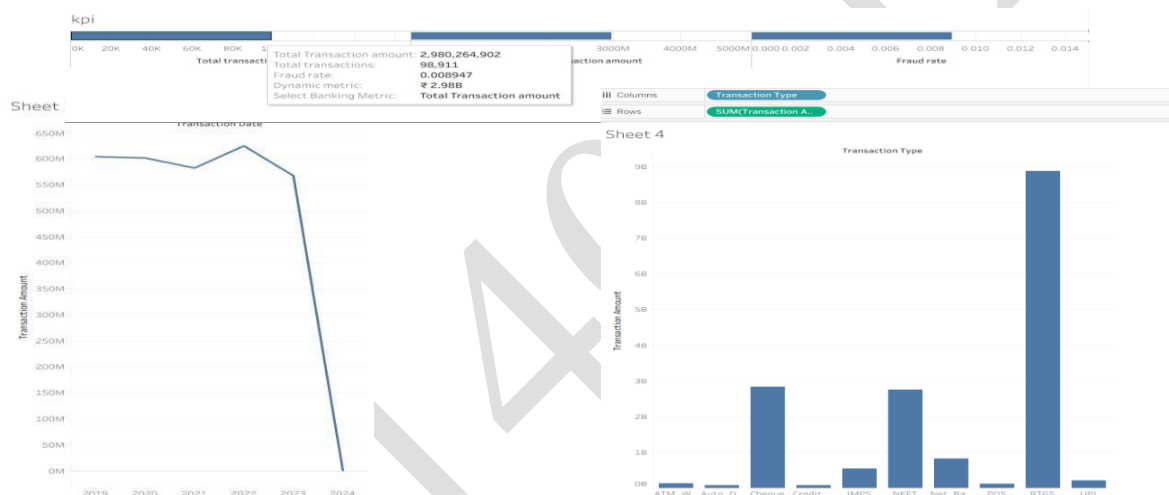
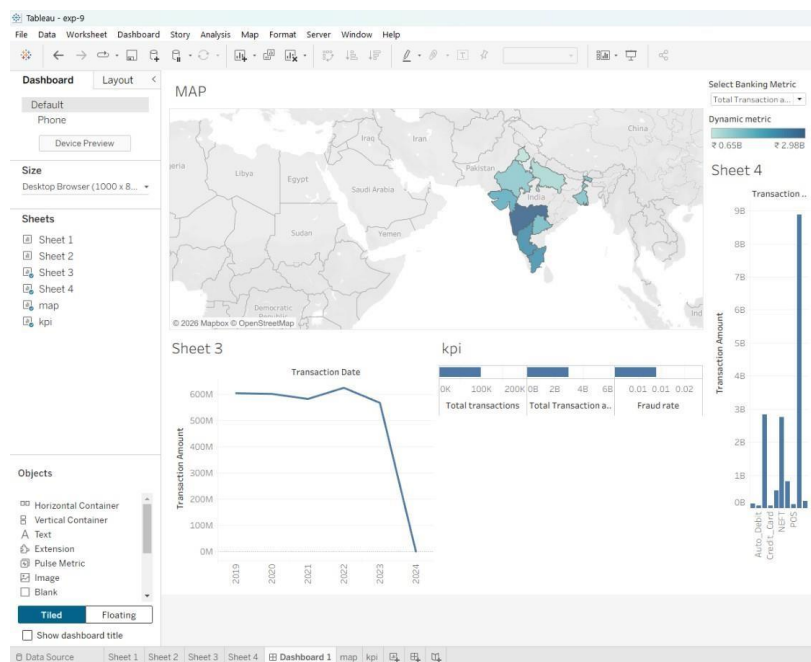
```
SUM(INT([Is Fraud])) / COUNT([Transaction Id])
```

- **Dynamic Metric Formula:**

Dynamic metric  banking_transactions (banking_analytics)

```
CASE [Select Banking Metric]
  WHEN 'Total Transaction amount' THEN [Total Transaction amount]
  WHEN 'Total transactions' THEN [Total transactions]
  WHEN 'Fraud rate' THEN [Fraud rate]
END
```

Step 7: Tableau Companion Dashboard & Interactive Story



- Worksheets:** Build KPI sheet, Hourly Trend Line Chart (using Dynamic Metric), Channel Breakdown Bar Chart, and State Map.
- Global Filters & Actions:**
 - Add Parameter control for Select Banking Metric to the dashboard.
 - Drag sheets onto a new Dashboard tab and click the **Use as Filter** funnel icon on the State Map visual.

Edit Filter Action

Name

State Map Drill-Through

Insert

Source Sheets

Dashboard 1

kpi

map

Sheet 3

Sheet 4

Run action on

Hover

Select

Menu

Single-select only

Target Sheets

Dashboard 1

kpi

map

Sheet 3

Sheet 4

Clearing the selection will

Keep filtered values

Show all values

Exclude all values

Filter

All fields

Selected fields

Source Field

Target Data Source

Target Field

Click to add

Remove

Cancel

OK

RESULT: